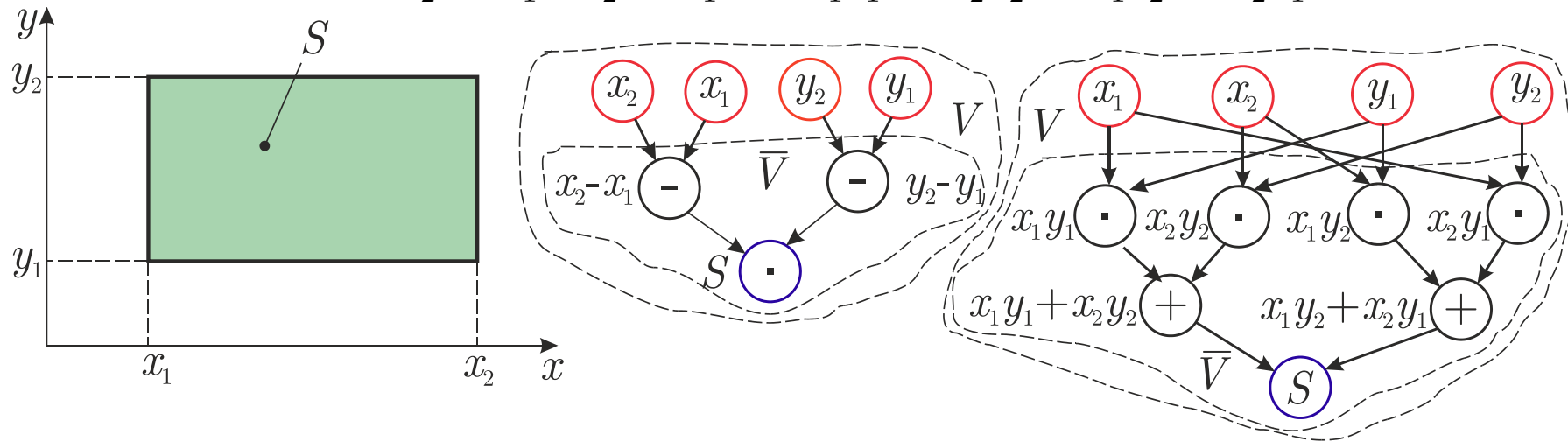


МОДЕЛИ ВЫЧИСЛЕНИЙ И АНАЛИЗ ЭФФЕКТИВНОСТИ ПАРАЛЛЕЛЬНЫХ АЛГОРИТМОВ

Гипотезы:

- Время выполнения любых вычислительных операций является одинаковым и равняется 1
- Передача данных между вычислительными устройствами происходит мгновенно

$$S = (x_2 - x_1)(y_2 - y_1) = x_1 y_1 + x_2 y_2 - x_1 y_2 - x_2 y_1$$



Множество операций и связи между ними задают ациклический орграф

$$G = (V, R), \quad V = \{1, 2, \dots, |V|\}, \quad R \subset V \times V$$

$r = (i, j) \in R \Leftrightarrow$ операция j использует результаты операции i

$\bar{V} \subset V$ – подмножество вершин, в которых выполняются вычислительные операции

Вершинам-источникам (вершинам ввода) $V \setminus \bar{V}$ сопоставлены операции ввода

$N = |V \setminus \bar{V}|$ – характерная размерность задачи

Вершинам-стокам сопоставлены операции вывода

Расписание для выполнения параллельной программы

$p \in \mathbb{N}$ – это количество процессоров, используемых для выполнения алгоритма

$t_i \in \mathbb{Z}$ – время начала выполнения операции в вершине i

P_i , $0 \leq P_i < p$ – номер процессора, выполняющего операцию в вершине i

Расписание: $H_p = \{(i, P_i, t_i) : i \in \bar{V}\}$, $t_i \in \mathbb{Z}$, $0 \leq P_i < p$

Для вершины ввода из $V \setminus \bar{V}$ время $t = 0$, но им не сопоставляются процессоры, то есть P_i не связывается с номером существующего процессора.

Требования реализуемости расписания

- $\forall i, j \in \bar{V}, i \neq j : t_i = t_j \Rightarrow P_i \neq P_j$, то есть один и тот же процессор не должен назначаться разными операциями в один и тот же момент времени
- $\forall i, j \in V, (i, j) \in R \Rightarrow t_j \geq t_i + 1$, то есть к назначенному моменту выполнения операции все необходимые данные уже должны быть вычислены.
- Если вершина $i \in \bar{V}$ непосредственно следует за вершиной ввода, то $t_i \geq 0$

Определение времени выполнения алгоритма

- Время выполнения параллельного алгоритма: $T_p(G, H_p) = \max_{i \in \bar{V}}(t_i + 1)$
- Для выбранной схемы используем расписание с минимальным временем исполнения алгоритма: $T_p(G) = \min_{H_p} T_p(G, H_p)$
- Выполняем подбор наилучшей вычислительной схемы $T_p = \min_G T_p(G)$
- Время выполнения последовательной версии алгоритма: $T_1 = |\bar{V}|$
- Минимальное время выполнения параллельного алгоритма при использовании неограниченного числа процессоров: $T_\infty = \min_{p \geq 1} T_p$
- На конкретной вычислительной системе минимальное время выполнения параллельного алгоритма может достигаться при использовании конечного числа процессоров
- $D(G)$ – длина максимального пути (Depth – глубина) в ациклическом орграфе.

Теорема 1. Минимально возможно время выполнения параллельного алгоритма определяется длиной максимального пути вычислительной схемы алгоритма, т.е.

$$T_{\infty} \geq D(G)$$

Доказательство. Вершинам ввода соответствует время $t=0$, и неотрицательные значения времени t соответствует началу выполнения операции в первой следующей за вершиной ввода вершине любого пути, в том числе и максимального пути. При дальнейшем движении вдоль любого пути, в том числе и вдоль максимального пути, для любого расстояния время начала выполнения операции в следующей вершине не менее чем на 1 больше, чем время начала выполнения операции в текущей вершине. Следовательно, время выполнения параллельного алгоритма не может быть меньше, чем длина максимального пути $D(G)$ в вычислительной схеме алгоритма, и будет совпадать с $D(G)$ в наилучшем случае. $\square$

Теорема 2. Пусть для некоторой вершины вывода в вычислительной схеме алгоритма существует путь из каждой $N = |V / \bar{V}|$ вершин ввода, и входная степень (т.е. число входящих дуг) вершины вычислительной схемы не превышает 2. Тогда: $D(G) \geq \log_2 N$

Доказательство. Для $x \in \mathbb{R}$ обозначим $\lceil x \rceil = \min\{n \mid n \geq x, n \in \mathbb{Z}\}$.

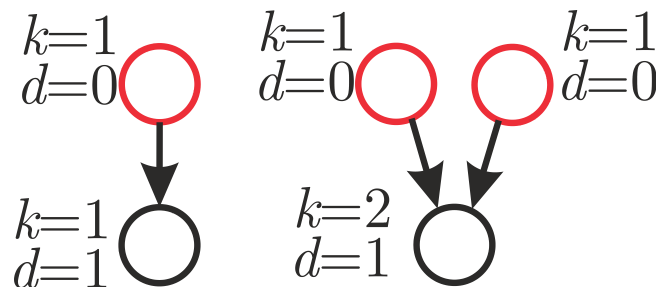
Считаем, что вершина j в графе G зависит от k входов, если существует k вершин ввода и путь от каждой из них к вершине j .

Пусть d_j — длина максимального пути от некоторой вершины ввода до вершины j . Для определённости полагаем, что вершина ввода зависит от одного входа $k=1$, и для неё $d=0$.

Докажем индукцией по k , что для любой вершины j , зависящей от k входов

$$d_j \geq \log_2 k \quad (1)$$

Утверждение (1) справедливо при $k=1,2$.



Предположим, что оно справедливо при $k \leq k_0$, и покажем его справедливость при $k = k_0 + 1$.

Пусть вершина j зависит от $k_0 + 1$ входов.

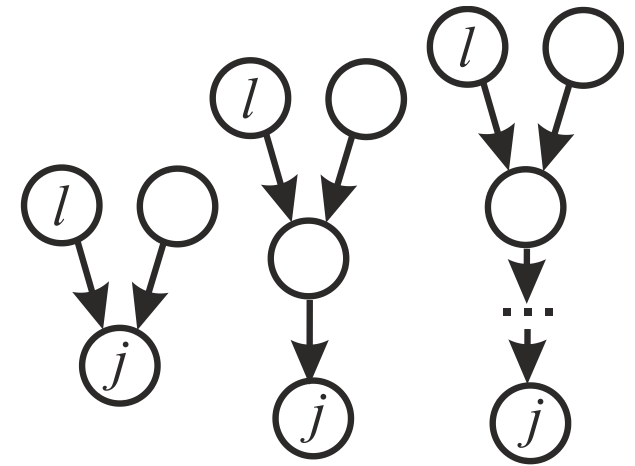
Так как граф ациклический и вершина j имеет не более 2 предшествующих ей вершин, то она имеет предшествующую ей вершину l , которая зависит не более чем от k_0 и не менее чем от $\lceil (k_0 + 1) / 2 \rceil$ входов.

Но тогда

$$d_j \geq d_l + 1 \geq \log_2 \lceil (k_0 + 1) / 2 \rceil + 1 \geq \log_2 (k_0 + 1),$$

т.е. (1) справедливо для любого $1 \leq k \leq N$. Как следует из (1)

$$D(G) \geq \max_{k=1, N} d_j \geq \log_2 N \quad \square$$



Теорема3. Пусть $c \in \mathbb{N}$, тогда при увеличении числа используемых процессоров в c раз при $q=cr$ справедливо неравенство $T_p \leq cT_q$.

Доказательство. Рассмотрим расписание, которое затрачивает время T_q с использованием q процессоров.

На каждом этапе может быть выполнено не более q операций и на это будет затрачиваться самое большее $q/p=c$ единиц времени на p процессорах.

Таким образом мы получили расписание для p процессоров, которое выполняется не более чем за cT_q единиц времени.

В свою очередь величина T_p не может превосходить времени выполнения данного расписания для p процессоров. $\square$

Теорема 4. Для любого количества $p \in \mathbb{N}$ используемых процессоров справедлива следующая верхняя оценка для времени выполнения параллельного алгоритма:

$$T_p < T_\infty + T_1 / p \quad (2)$$

Доказательство. Пусть H_∞ есть расписание для достижения минимального возможного времени выполнения T_∞ .

Для каждой итерации τ выполнения расписания H_∞ обозначим через n_τ количество операций, выполняемых в ходе итерации τ .

Если бы все операции выполнялись последовательно, то $T_1 = \sum_{\tau=1}^{T_\infty} n_\tau$ (3)

Расписание выполнения алгоритма с использованием p процессоров может быть построено следующим образом. Выполнение алгоритма разделим на T_∞ шагов; на каждом шаге τ следует выполнить все n_τ операций, которые выполнялись на итерации τ расписания H_∞ .

Выполнение этих операций может быть реализовано не более чем за $\lceil n_\tau / p \rceil$ итераций при использовании p процессоров. Как результат, время выполнения алгоритма T_p может быть оценено следующим образом:

$$T_p \leq \sum_{\tau=1}^{T_\infty} \left\lceil \frac{n_\tau}{p} \right\rceil \leq \sum_{\tau=1}^{T_\infty} \left(\frac{n_\tau}{p} + 1 \right) = \frac{T_1}{p} + T_\infty \quad (4)$$

Замечание. В каком-то смысле доказательство теоремы 4 даёт способ построения параллельного алгоритма. Сначала строится расписание без учёта ограниченности числа процессоров. Затем согласно схеме доказательства теоремы может быть построено расписание для конкретного количества процессоров.

Теорема 5. Времени выполнения алгоритма, которое сопоставимо с минимальным возможным временем T_∞ можно достичь при количестве процессоров порядка $p \sim T_1 / T_\infty$, а именно:

а) $p \geq \frac{T_1}{T_\infty} \Rightarrow T_p \leq 2T_\infty$

б) $p < \frac{T_1}{T_\infty} \Rightarrow \frac{T_1}{p} \leq T_p \leq 2\frac{T_1}{p}$

Доказательство.

а) Если $p \geq \frac{T_1}{T_\infty}$, то $\frac{T_1}{p} \leq T_\infty$, и утверждение немедленно следует из теоремы 4.

б) Если $p < \frac{T_1}{T_\infty}$, то $T_\infty < \frac{T_1}{p}$, и из теоремы 4 следует $T_p \leq 2\frac{T_1}{p}$.

Оценка $T_1 \leq pT_p$ следует из теоремы 3. $\square$

Рекомендации по правилам формирования параллельных алгоритмов:

- при выборе вычислительной схемы алгоритма должен использоваться граф G с минимальным возможным значением $D(G)$.
- для параллельного выполнения целесообразное количество процессоров оценивается величиной $p \sim T_1 / T_\infty$
- время выполнения параллельного алгоритма оценивается сверху вычислениями, приведёнными в теоремах 4 и 5.

Показатели эффективности параллельного алгоритма

Пусть N – характерная размерность задачи.

Ускорение (speedup), получаемое при использовании параллельного алгоритма для p процессоров, по сравнению с последовательным вариантом выполнения вычислений определяется величиной

$$S_p(N) = \frac{T_1(N)}{T_p(N)}$$

Эффективность использования параллельным алгоритмом процессоров при решении задачи определяется соотношением

$$E_p(N) = \frac{S_p(N)}{p} = \frac{T_1(N)}{p T_p(N)}$$

Из рассмотренных выше теорем следует, что теоретически в наилучшем случае

$$S_p(N) = p, \quad E_p(N) = 1$$

При определённых обстоятельствах ускорение может оказаться больше числа процессоров, и в этом случае говорят о **сверхлинейном (superlinear)** ускорении. Причины:

- Кластерные системы с распределенной памятью: При решении задачи на одном процессоре оказывается недостаточно оперативной памяти для хранения всех обрабатываемых данных, и, как результат, необходимым становится использование более медленной внешней памяти.
- Симметричные мультипроцессорные системы: параллельный алгоритм для некоторых значений характерной размерности N задачи может лучше использовать ресурсы кэш-памяти имеющихся процессоров.
- Различие вычислительных схем последовательного и параллельного методов.

Стоимость вычислений: произведение времени параллельного решения задачи и числа используемых процессоров

$$C_p = pT_p$$

Стоимостно-оптимальный параллельный алгоритм: стоимость которого является пропорциональной времени выполнения T_1 наилучшего последовательного алгоритма.

Параллельный алгоритм суммирования $\sum_{j=1}^N x_j$ с использованием фиксированного числа $p \in \mathbb{N}$ процессоров

$$T_1 = N - 1, \quad T_p = \frac{N - 1}{p} + p - 1$$

Ускорение, эффективность и стоимость параллельного алгоритма

$$S_p(N) = \frac{T_1(N)}{T_p(N)} = \frac{N - 1}{(N - 1) / p + p - 1} \xrightarrow{N \rightarrow \infty} p$$

$$E_p(N) = \frac{S_p(N)}{p} = \frac{N - 1}{N - 1 + p(p - 1)} \xrightarrow{N \rightarrow \infty} 1$$

$$C_p(N) = p T_p(N) = N - 1 + p(p - 1)$$

- При $N \rightarrow \infty$ данный алгоритм является стоимостно-оптимальным.
- С ростом характерной размерности N решаемые задачи распараллеливаются более эффективно
- То же самое справедливо и для распараллеливания свертывания достаточно длинной последовательности $x_j \in X$, $j = 1, 2, \dots, N$ ассоциативной коммутативной бинарной операцией $\circ$

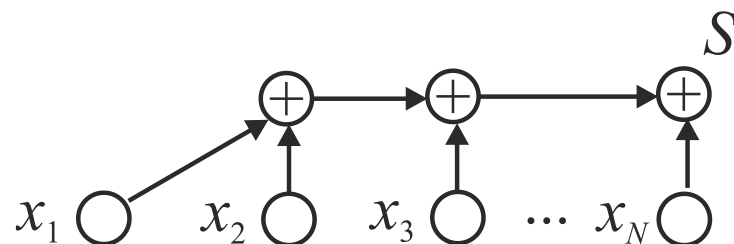
$$A = x_1 \circ x_2 \circ x_3 \circ \dots \circ x_{N-2} \circ x_{N-1} \circ x_N$$

Концепция неограниченного параллелизма

$$S = \sum_{j=1}^N x_j$$

Последовательный алгоритм суммирования суть

$$S \leftarrow x_1, S \leftarrow S + x_2, \dots, S \leftarrow S + x_N \quad (2.17)$$

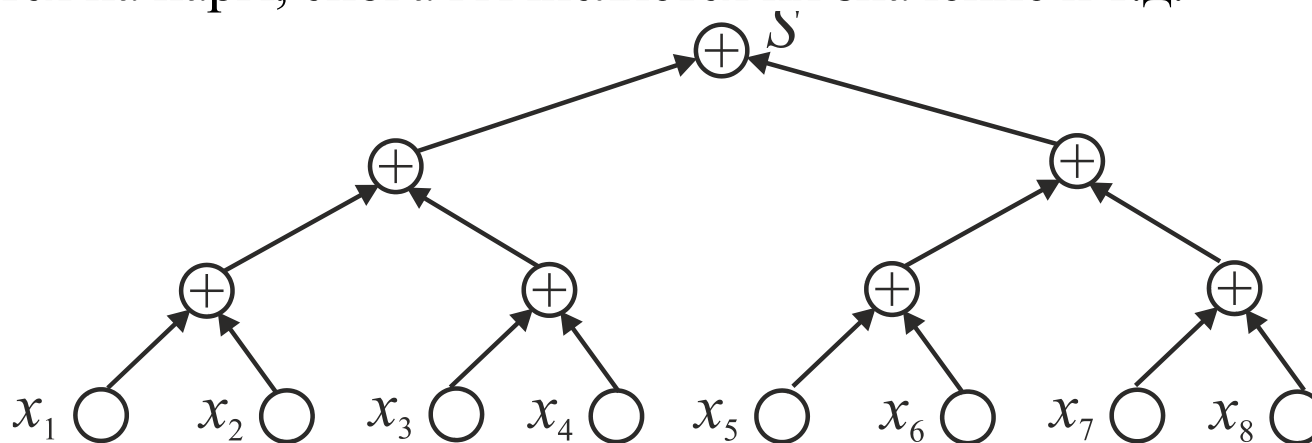


$$D(G) = N - 1$$

$$T_1 = N - 1$$

Каскадная схема суммирования

На первой итерации каскадной схемы суммирования все слагаемые разбиваются на пары, и для каждой пары вычисляется сумма их значений. Далее все полученные суммы снова разбиваются на пары, снова вычисляется их значение и т.д.



Пусть $N = 2^n$. На каждой итерации число операций уменьшается в 2 раза, число итераций $n = \log_2 N$, «глубина» графа «операции-операнды» суть $D(G) = n$.

На первой итерации выполняется $N/2$ операций, и $p = N/2$ процессоров.

Общее количество действий $2^{n-1} + 2^{n-2} + \dots + 2 + 1 = 2^n - 1 = N - 1$.

т. к. $D(G) = n$, то время выполнения каскадной схемы $T_p = D(G) = n = \log_2 N$

Ускорения и эффективность:

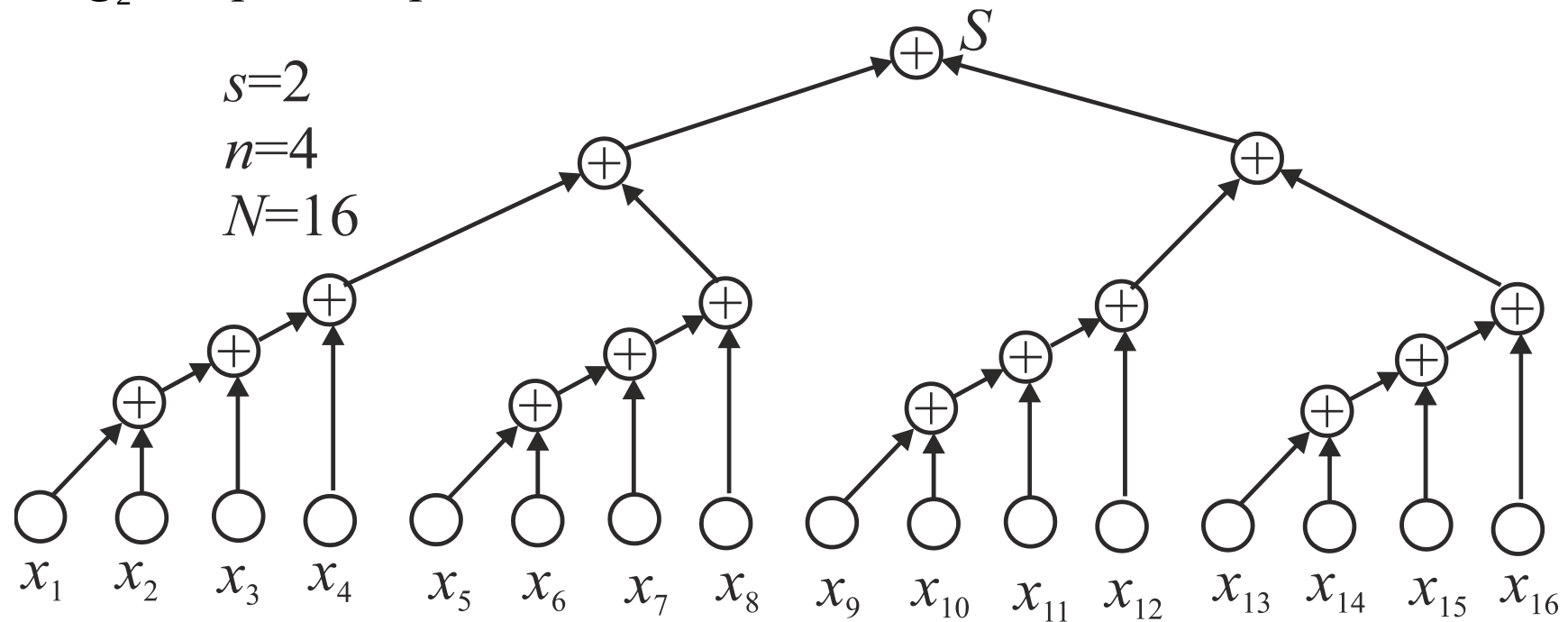
$$S_p(N) = \frac{T_1}{T_p} = \frac{N-1}{\log_2 N} \xrightarrow{N \rightarrow \infty} \infty, \quad E_p(N) = \frac{S_p(N)}{p} = \frac{2}{N} \frac{N-1}{\log_2 N} \xrightarrow{N \rightarrow \infty} 0$$

Те же самые оценки: для свертывания достаточно длинной последовательности ассоциативной коммутативной бинарной операцией

Модифицированная каскадная схема

Полагаем $N = 2^n$, $n = 2^s$.

- На первом этапе вычислений все суммируемые значения подразделяются на $N / \log_2 N$ групп, в каждой из которых содержится $\log_2 N$ элементов. Далее для каждой группы вычисляются сумма значений при помощи последовательного алгоритма суммирования. Вычисления в каждой группе могут выполняться независимо друг от друга, т.е. параллельно, и для этого потребуется как минимум $p = N / \log_2 N$ процессоров.



- На втором этапе для полученных $N / \log_2 N$ сумм отдельных групп применяется каскадная схема.

Для выполнения первого этапа потребуется выполнить $\log_2 N$ параллельных операций на $p = N / \log_2 N$ процессорах.

Для выполнения второго этапа потребуется

$$\log_2 \frac{N}{\log_2 N} = \log_2 N - \log_2 \log_2 N \leq \log_2 N$$

параллельных операций выполненных на $\frac{p}{2} = \frac{N}{2 \log_2 N}$ процессорах.

Как результат: $T_p = 2 \log_2 N$

Ускорение и эффективность параллельного алгоритма суть

$$S_p(N) = \frac{T_1}{T_p} = \frac{N-1}{2 \log_2 N} \xrightarrow{N \rightarrow \infty} \infty, \quad E_p(N) = \frac{S_p}{p} = \frac{\log_2 N}{N} \frac{N-1}{2 \log_2 N} \xrightarrow{N \rightarrow \infty} \frac{1}{2}$$

Модифицированный каскадный алгоритм является стоимостно-оптимальным, поскольку

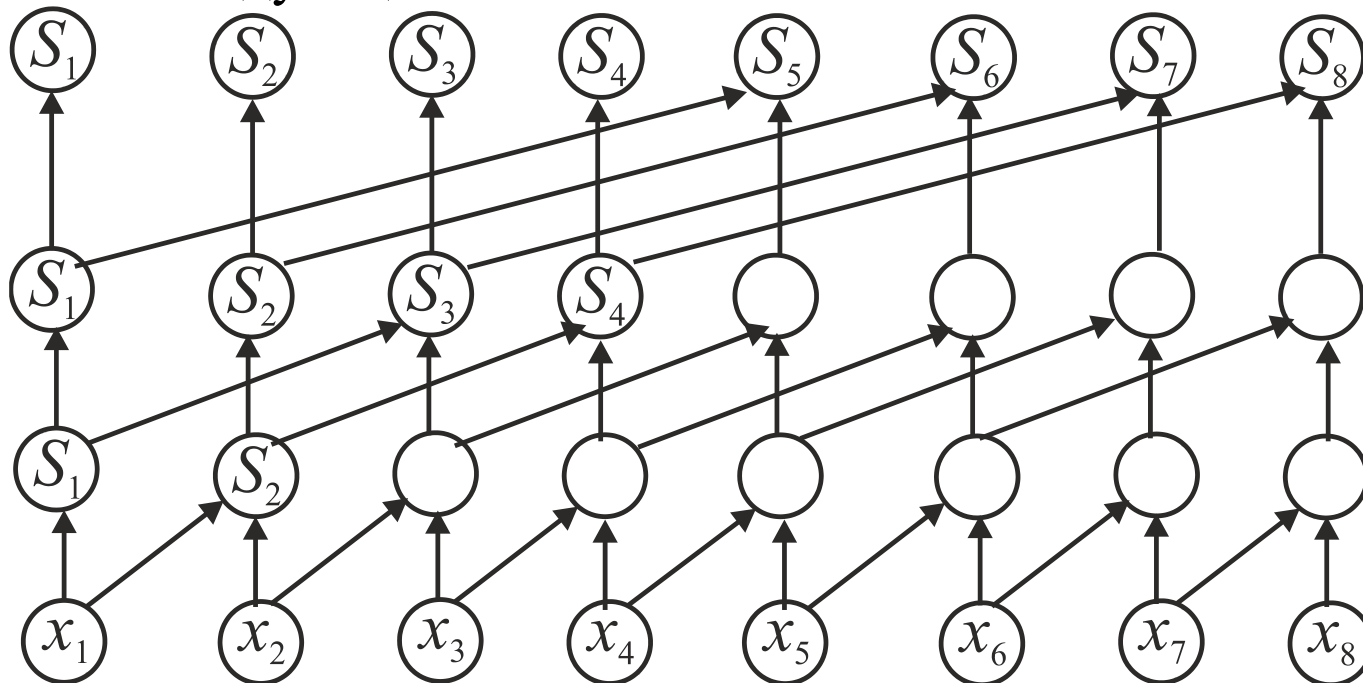
$$C_p = p T_p = \frac{N}{\log_2 N} 2 \log_2 N = 2N$$

Те же самые оценки: для свертывания достаточно длинной последовательности ассоциативной коммутативной бинарной операцией

Поиск всех частных сумм.

$$S_1 = x_1, S_2 = x_1 + x_2, \dots, S_N = x_1 + x_2 + \dots + x_N$$

Алгоритм, обеспечивающий получение результатов за $\log_2 N$ параллельных операций, состоит в следующем.



- На первом шаге создается копия $\mathbf{S} = \mathbf{x}$ вектора суммируемых значений.
- Далее на каждой итерации суммирования с номером k , $1 \leq k \leq \log_2 N$, формируется вспомогательный вектор $\mathbf{Q}$, путем сдвига вправо вектора $\mathbf{S}$ на 2^{k-1} позиций . (освободившиеся слева позиций в векторе $\mathbf{Q}$ больше не нужны и можно формально полагать, что они заполнены нулем).

- Итерация завершается параллельной операцией прибавления к элементам вектора S элементов вектора Q , начиная с позиции $2^{k-1} + 1$, т.е.

$$Q_{2^{k-1}+j} \leftarrow S_j, \quad j = 1, 2, \dots, N - 2^{k-1}$$

$$S_j \leftarrow S_j + Q_j, \quad j = 2^{k-1} + 1, \dots, N$$

$$k = 1, 2, \dots, \log_2 N$$

Т. е. время выполнения параллельного алгоритма

$$T_p(N) = \log_2 N.$$

Для этого требуется $p=N-1$ процессоров, и число операций

$$(N-1) + (N-2) + (N-2^2) + \dots + (N/2) \sim N \log_2 N$$

что ощутимо больше, чем в последовательном алгоритме.

Ускорение и эффективность параллельного алгоритма

$$S_p(N) = \frac{N-1}{\log_2 N} \xrightarrow{N \rightarrow \infty} \infty, \quad E_p(N) = \frac{1}{\log_2 N} \xrightarrow{N \rightarrow \infty} 0$$

Оценка максимально достижимого параллелизма. Закон Амдала

Позволяет оценить максимальное достижимое ускорение в зависимости от относительной доли нераспараллеливаемых операций в последовательной версии алгоритма.

Как правило, некоторый последовательный алгоритм содержит чередующиеся последовательности операций, которые можно выполнить лишь последовательно, и последовательности операций, для которых можно организовать параллельное выполнение.

Пусть $m^{(1)}, m^{(2)}, m^{(3)}, \dots$ – длины последовательности операций, которые можно выполнить лишь последовательно; $n^{(1)}, n^{(2)}, n^{(3)}, \dots$ – длины последовательности операций, для которых можно организовать параллельное выполнение.

Соответственно:

$m = \sum_j m^{(j)}$ – общее количество операций, которые можно выполнять лишь последовательно;

$n = \sum_j n^{(j)}$ – общее количество операций, для которых можно организовать параллельное выполнение ;

$f = m / (m + n)$ – доля последовательных вычислений в последовательной версии алгоритма.

Теорема 6 (Закон Амдала). Пусть f есть доля последовательных вычислений в последовательной версии применяемого алгоритма обработки данных. Тогда наибольшее ускорение процесса вычислений при использовании p процессоров определяется величиной

$$S_p^{(\max)} = \frac{1}{f + (1-f)/p} \xrightarrow{p \rightarrow \infty} S^{(\max)} = \frac{1}{f}$$

Доказательство. Время выполнения последовательной версии алгоритма $T_1 = m + n$. В параллельной версии алгоритма время выполнения операций, которые можно выполнить лишь последовательно, останется тем же самым, т.е. m . Пусть $T_p^{(j)}$ – время выполнения последовательности $n^{(j)}$ операций исходного последовательного алгоритма, для которых организовано параллельное выполнение на p процессорах; $T_p^* = \sum_j T_p^{(j)}$ – общее время выполнение n операций исходного алгоритма, для которых организовано параллельное выполнение. Из теоремы 3 следует: $T_p^{(j)} \geq n^{(j)} / p \Rightarrow T_p^* \geq n / p$.

Тогда общее время выполнения параллельного алгоритма и ускорения допускают оценку: $T_p = m + T_p^* \geq m + n / p$

$$S_p = \frac{T_1}{T_p} \leq \frac{m+n}{m+n/p} = \frac{1}{\frac{m}{m+n} + \frac{(m+n)-m}{m+n} \frac{1}{p}} = \frac{1}{f + (1-f)/p} \quad (2.34)$$

Замечание. Для рассмотренного в первой лекции преобразования алгоритма суммирования для параллельного суммирования на p процессорах доля последовательных вычислений $f = p / N \xrightarrow{N \rightarrow \infty} 0$. Это означает, что, как правило, величина f значительно уменьшается с ростом характерной размерности задачи, и следовательно, распараллеливание более крупных блоков алгоритма, как правило приводит к лучшим результатам.

Замечание. Тем не менее, объективная оценка величины f зависит от особенностей используемой вычислительной системы и может быть проанализирована на основании сравнения время выполнения последовательной и параллельной версии алгоритма для различных значений характерной размерности задачи.

Замечание. Более эффективное использование параллельной версией программы значительных объемов кэш-памяти современных процессоров в ряде случаев приводит к сверхлинейному ускорению, превышающему число процессоров.

Закон Густавсона- Барсиса

Позволяет оценить максимально достижимое ускорение, исходя из относительной доли последовательных расчетов в выполняемых параллельных вычислениях.

Пусть N - характеристическая размерность задачи, $\tau(N)$ и $\pi(N)$ – суть времена выполнения последовательной и параллельной частей выполняемых вычислений соответственно, т.е.

$$T_1 = \tau(N) + \pi(N), T_p = \tau(N) + \pi(N) / p$$

Доля последовательных вычислений в параллельной версии алгоритма

$$g = \frac{\tau(N)}{\tau(N) + \pi(N) / p}$$

но тогда ускорение вычислительного процесса суть

$$\begin{aligned} S_p(N) &= \frac{T_1}{T_p} = \frac{\tau(N) + \pi(N)}{\tau(N) + \pi(N) / p} = \frac{p(\tau(N) + \pi(N) / p) - (p-1)\tau(N)}{\tau(N) + \pi(N) / p} = \\ &= p + (1-p)g \end{aligned}$$

Оценка ускорения вычислений по Густовсону- Барсису применяется тогда, когда, в силу большой трудоёмкости задачи, ее решения на однопроцессорной системе становится невозможным, например из-за отсутствия оперативной памяти.

Строгая развертка ориентированного ациклического графа и его ярусно- параллельная форма

Для фиксированного ациклического ориентированного графа G , $R \subset V \times V$ строгой разверткой называется такая функция $f : V \rightarrow \mathbb{R}$, что

$$\forall i, j \in V : \exists r = (i, j) \in R \Rightarrow f(j) > f(i)$$

Если ориентированный граф содержит цикл, то строгой развертки для него не существует.

Если $f_{1,2}$ – строгие развертки, то функции $Const \cdot f_1$, $Const > 0$ и $f_1 + f_2$ – также строгие развертки.

Пусть $V_i \subset V, i = 1, 2, \dots$ – поверхности уровня для развертки $f : V \rightarrow \mathbb{R}$, т.е.

$$\forall i_1, i_2 \in V_i \quad f(i_1) = f(i_2)$$

- Разбиение вершин V графа G на поверхности уровня V_i развертки f определяет т.н. ярусно-параллельную форму ориентированного ациклического графа в том смысле, что, если $i_1 \in V_i$, $j_1 \in V_j$, и существует дуга графа $r = (i_1, j_1) \in R$, то отсюда следует $i \neq j$, и можно перенумеровать поверхности уровня V_i так, что $j > i$.
- Каждое из множеств V_i называется ярусом, i – его номером, количество вершин $|V_i|$ в ярусе – его шириной.
- Количество ярусов в ярусно-параллельной форме называется ее высотой, а максимальная ширина ее ярусов – шириной ярусно-параллельной формы.

- Так как никакие две вершины в максимальном пути графа G не могут лежать на одной и той же поверхности уровня развертки, минимальная высота параллельной формы ациклического ориентированного графа не может быть меньше $D(G) + 1$.
- Для ярусно-параллельной формы графа существенным является тот факт, что для вершин на одной и той же поверхности уровня нет соединяющих их дуг, и, более того, нет путей в графе G , соединяющих данные вершины.
- Следовательно, операции, соответствующие вершинам на одной и той же поверхности уровня, независимы, и могут быть выполнены параллельно.

Обобщенные развертки и выделение независимых подзадач

Для фиксированного ориентированного ациклического графа $G = (V, R)$, $R \subset V \times V$ обобщенной разверткой называется такая функция его вершин $f : V \rightarrow \mathbb{R}$, что

$$\forall i, j \in V : \exists r = (i, j) \in R \Rightarrow f(j) \geq f(i) \quad (5)$$

Если $f_1, f_2 : V \rightarrow \mathbb{R}$ – обобщенные развертки, то функции

$$Const \cdot f_1, Const > 0, f_1 + f_2, \max(f_1, f_2), \min(f_1, f_2) \quad (6)$$

– также обобщенные развертки.

Замечание. Граф «операции-операнды» является ориентированным ациклическим, однако для обобщенных разверток свойство ацикличности несущественно.

Теорема 7. Пусть для ориентированного графа известны обобщенные развертки f_1 и f_2 , $\alpha > 0$, $\beta > 0$ и $f = \alpha f_1 + \beta f_2$. Тогда, если для $i, j \in V$

$$f(i) = f(j) \quad (7)$$

но

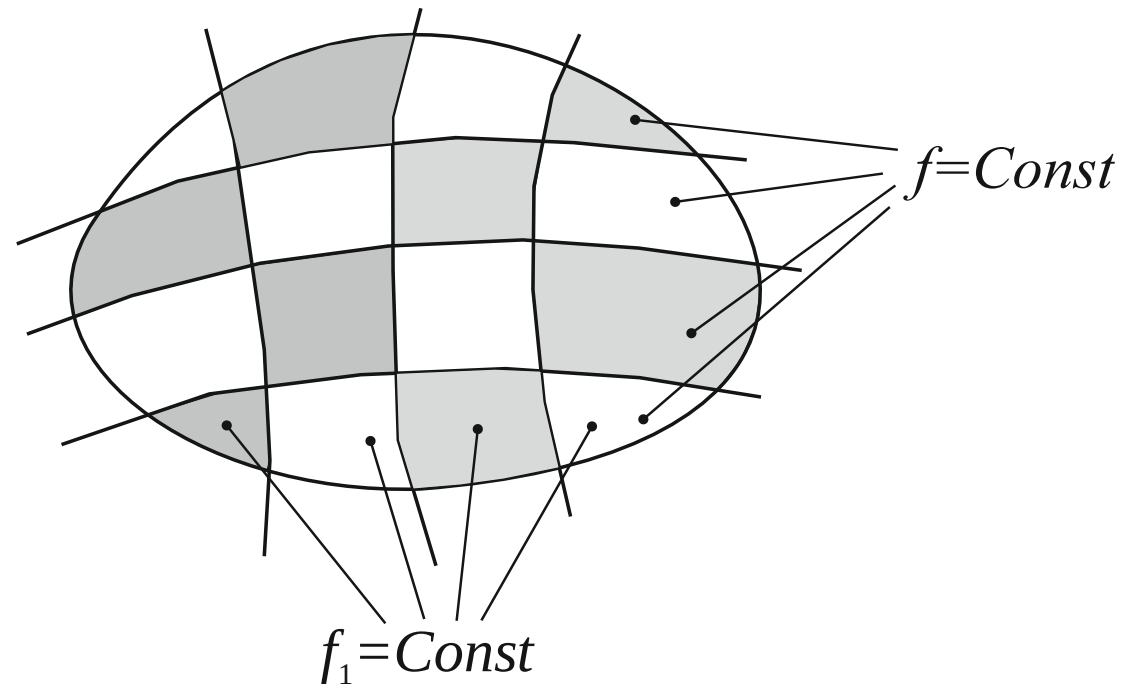
$$f_1(i) \neq f_1(j) \quad (8)$$

то вершины i и j не могут быть связаны путем в ориентированном графе G .

Доказательство. Пусть $f_1(i) > f_1(j)$. Но тогда $f_2(i) < f_2(j)$. Т.к. по определению значение обобщенной развертки не может убывать при движении вдоль пути в ориентированном графе G , то не существует ни пути из вершины i в вершину j , ни пути из вершины j в вершину i .

Аналогичный вывод имеет место и в предположении $f_1(i) < f_1(j)$ $\square$

Теорема 7 позволяет выделять множества вершин в графе алгоритма, например, множества V_1 и V_2 , таким образом, что операции, выполняемые в вершинах из множества V_1 , взаимно-независимы с операциями, выполняемыми в вершинах из множества V_2 . В самом деле, сначала разбиваем вершины графа на поверхности уровня функции f , в пределах каждой из которых выполняются равенства (7). Затем возьмем любую из разверток f_1 или f_2 , для определенности, f_1 , и в соответствии с ее поверхностями уровня разобьем на множества поверхности уровня развертки f . Тогда, если множества $V_1, V_2 \in V$ лежат на одной поверхности уровня развертки f , но на разных поверхностях уровня развертки f_1 , то $\forall i \in V_1, j \in V_2 \ f(i) = f(j)$ и $f_1(i) \neq f_1(j)$. Но это означает отсутствие в графе «операции-операнды» путей из V_1 в V_2 и из V_2 в V_1 , т.е. операции, соответствующие вершинам из V_1 и из V_2 , взаимно-независимы, и могут быть выполнены параллельно.



- Более подробное разбиение множества вершин графа «операции-операнды» на подмножества, не связанные путями в графе, т.е. на взаимно-независимые множества операций, может осуществляться до тех пор, пока не останется неиспользованной одна из тех разверток, которые образовали исходную линейную комбинацию разверток.
- Подобный параллелизм, определяемый поверхностями уровня обобщенных разверток, называется «скошенным», и часто применяется, например, при распараллеливании вложенных циклов. При этом граф «операции-операнды» фрагмента алгоритма, соответствующего вложенному циклу, удобно формально расположить в пространстве, размерность которого равна глубине цикла.

ПРИНЦИПЫ РАЗРАБОТКИ ПАРАЛЛЕЛЬНЫХ АЛГОРИТМОВ

Синхронный и асинхронный способы обработки данных

- Архитектура SIMD на аппаратном уровне предполагает т.н. синхронную параллельную мультиобработку данных.
- Архитектура MIMD предполагает, что параллельная программа реализуется в некоторых процессах либо потоках ОС, каждый из которых может выполняться со своей скоростью, т.е. вообще говоря, асинхронно – по причине разной загруженности отдельных процессоров мультипроцессора либо разных вычислительных характеристик отдельных узлов мультипроцессора.

Синхронные и асинхронные параллельные процессы

- Асинхронный параллельный процесс — это процесс, состояние которого не зависит от состояния другого параллельного процесса.
- Синхронный параллельный процесс — это параллельный процесс, состояние которого зависит от состояния взаимодействующего с ним параллельного процесса. В этом случае, реализуется синхронизация взаимодействующих процессов или потоков ОС, выполняющихся в процессах параллельной программы, и можно говорить о синхронных параллельных вычислениях.

Основные виды синхронизации

- Взаимное исключение обеспечивает в текущий момент времени работу с некоторым критичным ресурсом лишь одного процесса или потока ОС.
- Условная синхронизация означает ожидание процесса или потока ОС вплоть до выполнения некоторого условия.

Параллельные вычисления реализуются в основном либо в виде программ, параллельных по данным, в которых все процессы либо потоки ОС выполняют одни и те же действия, но каждый с собственной частью данных, либо в виде программ, параллельных по задачам, в которых различные процессы либо потоки ОС решают различные задачи.

Парадигмы (т.е. модели решений) параллельного программирования

- Итеративный параллелизм. Используется, когда в программе есть несколько процессов или потоков, каждый из которых содержит один или несколько достаточно длинных циклов. Таким образом, каждый процесс (поток) является итеративной программой. Процессы (или потоки) программы работают совместно над решением одной задачи. Они взаимодействуют и синхронизируются при помощи разделяемых переменных или передачи сообщений. Итеративный параллелизм чаще всего встречается в научных вычислениях, выполняемых на нескольких процессорах.
- Рекурсивный параллелизм. Рекурсивную процедуру (т.е функцию C++) можно реализовать с помощью параллелизма, если она содержит несколько независимых рекурсивных вызовов самой себя. Два вызова процедуры являются независимыми, если а) процедура не изменяет значений глобальных переменных б) фактические выходные параметры и результирующие переменные (если они есть) являются различными переменными. Пример реализации: некоторая процедура (функция C++) разбивает массив на две части, и затем дважды вызывает себя, сначала для левой части, потом для правой.

- Производители и потребители. Это взаимодействующие процессы, они часто объединяются в конвейер, в котором каждый процесс является фильтром, потребляющим данные своего предшественника и генерирующим данные для своего последователя. По сути, это аналог конвейерной модели вычислений, в которой каждый процесс играет роль ступени конвейера.

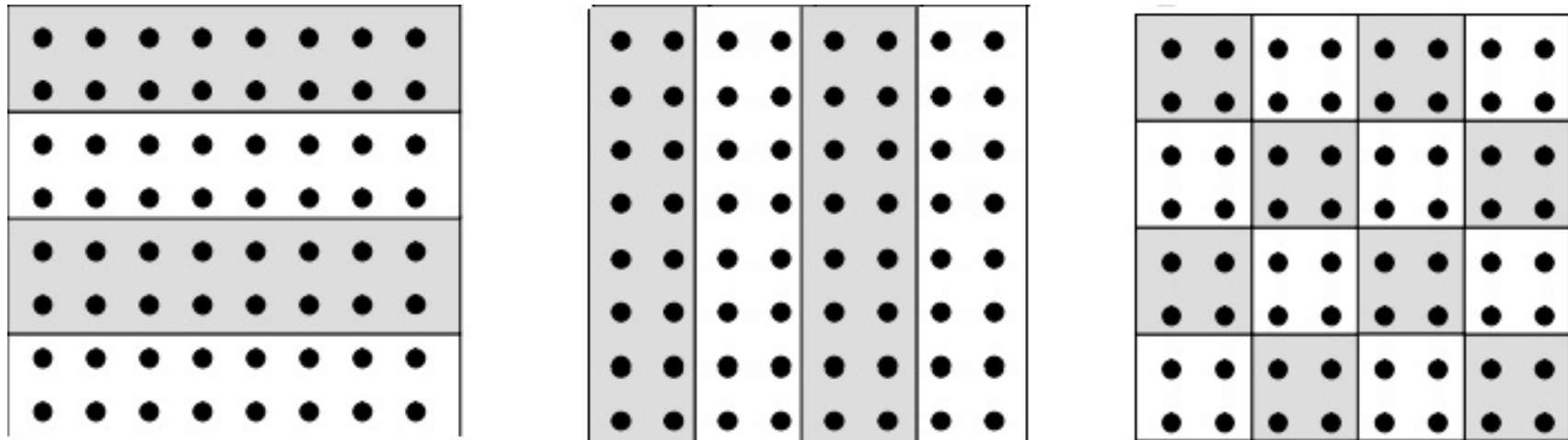


- Клиенты и серверы. Достаточно распространенная модель взаимодействия, наиболее часто используется в распределенных системах, от локальных сетей до Internet. В рамках данной модели клиентский процесс (или поток ОС) запрашивает сервис и ждет ответа. Сервер ожидает запросов от клиентов, а затем действует в соответствии с этими запросами. Сервер может быть реализован как одиночный процесс, который не может обрабатывать одновременно несколько клиентских запросов, или как многопоточная программа (при необходимости параллельного обслуживания запросов)
- Взаимодействующие равные. Данная парадигма параллельного программирования в основном применяется в программах, в которых несколько процессов для решения задачи выполняют один и тот же код и обмениваются сообщениями. В основном применяется для реализации параллельных программ, работающих на мультикомпьютерах с распределенной памятью, в особенности при итеративном параллелизме.

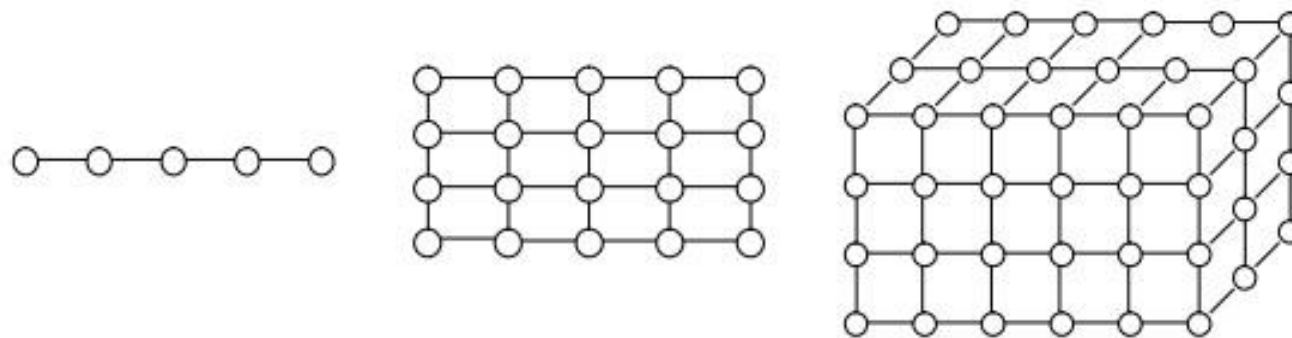
Этапы разработки параллельных алгоритмов

Разделение вычислений на независимые части.

- Выбор способа разделения вычислений на независимые части основывается на анализе вычислительной схемы решения исходной задачи. Основные требования при этом состоят в обеспечении по возможности равного объема вычислений в выделяемых подзадачах и минимума информационных зависимостей между этими подзадачами. В общем случае, проведение анализа и выделение подзадач является достаточно сложной проблемой, однако ситуация достаточно легко решается для двух типов вычислительных схем:
- Для большого класса задач вычисления сводятся к выполнению однотипной обработки большого набора данных. Например: матричные вычисления, численные методы решения уравнений в частных производных и т.д. В этом случае говорят, что существует параллелизм по данным, и выделение подзадач сводится к распределению имеющихся данных. Рассмотрим, например, поиск максимального значения в матрице A . При этом матрица A может быть разделена на отдельные строки/столбцы (или последовательные группы строк/столбцов) - ленточная схема разделения данных или на прямоугольные наборы элементов - блочная.



- Для большого количества решаемых задач разделение вычислений по данным приводит к возникновению одно-, двух- и трехмерных наборов подзадач, в которых информационные связи существуют только между ближайшими соседями (также схемы обычно именуются решетками)

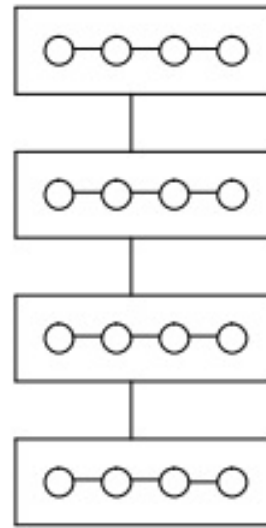


- Для другой части задач вычисления могут состоять в выполнении операций над одним и тем же набором задач - в этом случае говорят о существовании функционального параллелизма. Часто функциональная декомпозиция может быть использована для организации конвейерной обработки данных.
- Важным является вопрос о выборе нужного уровня декомпозиции вычислений. Формирование максимального количества подзадач обеспечивает использование предельно достижимого уровня параллелизма решаемой задачи, однако затрудняет анализ параллельных вычислений. Использование при декомпозиции вычислений только достаточно крупных подзадач приводит к ясной схеме параллельных вычислений, однако может затруднить эффективное использование достаточно большого количества процессоров. Возможное разумное сочетание этих двух подходов может состоять в использовании в качестве конструктивных элементов декомпозиции только тех задач, для которых методы параллельных вычислений хорошо известны.
- Для оценки корректности этапа разделения вычислений на независимые части можно воспользоваться следующим контрольным списком вопросов:
 - а) Увеличивает ли объем вычислений и необходимый объем памяти выполненная декомпозиция на подзадачи?
 - б) Возможна ли при выборочном способе декомпозиции равномерная загрузка всех имеющихся элементов в вычислительной системе?
 - с) Достаточно ли выделенных подзадач для эффективной загрузки имеющихся процессоров с учетом возможности увеличения их количества?

Выделение информационных зависимостей.

- Этапы выделения подзадач и информационных зависимостей очень сложно разделить. Выделение подзадач должно происходить с учетом возникающих информационных связей. После анализа объема и частоты необходимых информационных обменов между подзадачами может потребоваться повторение этапа разделения вычислений.
- При проведении анализа информационных зависимостей между задачами различают:
 - Локальные и глобальные схемы информационного взаимодействия. Для локальных схем в каждый момент времени информационная зависимость существует лишь между небольшим числом подзадач, располагаемых, или правило, на соседних вычислительных элементах. Для глобальных зависимостей информационное взаимодействие имеет место для всех подзадач.
 - Структурные и произвольные способы взаимодействия. Для структурных способов организация взаимодействий приводит к формированию некоторых стандартных схем коммуникации (в виде кольца, произвольной решетки и т.д.), тогда как для произвольных структур взаимодействия схема информационных зависимостей не носит характер какой-либо однородности.
 - Статические и динамические схемы информационной зависимости. Для статических схем моменты и участники информационного взаимодействия формируются на этапе проектирования и разработки параллельных программ. Для динамического варианта взаимодействия структура зависимостей определяется в ходе выполняемых вычислений.

- Например, для задачи поиска максимального значения в матрице при использовании в качестве базовых элементов подзадач поиска максимальных элементов в отдельных строках исходной матрицы структура информационных связей имеет следующий вид:



Для оценки правильности этапа выделения информационных зависимостей можно воспользоваться следующим списком вопросов:

- а) Соответствует ли вычислительная мощность подзадач интенсивности их информационных взаимодействий?
- б) Одинакова ли интенсивность информационных взаимодействий для разных подзадач?
- с) Является ли схема информационного взаимодействия локальной?
- д) Не препятствует ли выявленная информационная зависимость параллельному решению подзадач?

Масштабирование набора подзадач

- Масштабирование разработанной вычислительной схемы параллельных вычислений проводится в случае, если количество имеющихся подзадач отличается от числа планируемых к использованию вычислительных элементов.
- Для сокращения количества подзадач необходимо выполнять укрупнение (агрегацию) вычислений. Применяемые здесь правила совпадают с рекомендациями начального этапа выделения подзадач - определяемые подзадачи, как и ранее, должны иметь одинаковую вычислительную сложность, а объем и интенсивность информационных взаимодействий между подзадачами должны оставаться на минимально возможном уровне. Т.е. первыми претендентами на объединение являются подзадачи с высокой степенью информационной зависимости.
- При недостаточном количестве имеющихся в наборе подзадач для загрузки всех доступных к использованию вычислительных элементов необходимо, наоборот, выполнить детализацию (декомпозицию) вычислений. Как правило, выполнение декомпозиции не вызывает затруднений, если для базовых задач методы параллельных вычислений известны.
- Пример агрегации: объединение строк матрицы в группы.
- Пример декомпозиции: разбиение групп строк матрицы на блоки.

- Т.е. выполнение этапа масштабирования вычислений должно свестись к разработке правил агрегации и декомпозиции подзадач, которые должны параметрически зависеть от числа процессоров.
- В типовых ситуациях использование современных технологий параллельного программирования, например, OpenMP или MPI, автоматически обеспечивает хорошее масштабирование типовых вычислительных процессов по числу имеющихся процессоров.
- Для оценки правильности выполнения данного этапа служат следующие контрольные вопросы:
 - a) Не ухудшится ли локальность вычислений после масштабирования имеющегося набора подзадач?
 - b) Имеют ли подзадачи после масштабирования одинаковую вычислительную и коммуникационную сложность?
 - c) Соответствует ли количество задач числу имеющихся процессоров?
 - d) Зависят ли параметрически правила масштабирования от количества процессоров?

Распределение подзадач между процессорами.

- Распределение подзадач между процессорами является завершающим этапом разработки параллельного метода.
- Управление распределением нагрузки для процессоров возможно в основном для мультикомпьютеров с распределенной памятью, т.к. для мультипроцессоров с общей памятью распределение нагрузки обычно выполняется операционной системой автоматически.
- Этап распределения подзадач между процессорами является избыточным, если количество подзадач совпадает с числом имеющихся процессоров.
- Основной показатель успешности выполнения данного этапа – эффективность использования процессов параллельным алгоритмом. Для этого необходимо обеспечить равномерное распределение вычислительной нагрузки между процессорами и минимизировать количество информационных взаимодействий, существующих между ними.
- Требования минимизации информационных обменов между процессорами и обеспечения равномерной загрузки процессов вычислительной системы являются взаимно-противоречивыми. Например, в последовательной программе информационные взаимодействия в принципе отсутствуют.

- Решение вопросов балансировки вычислительной нагрузки значительно усложняется, если схема вычислений может изменяться в ходе решения задачи. Причиной этого могут быть, например, неоднородные сетки при конечно-элементном численном моделировании задач математической физики, разреженность матриц, использование адаптивных алгоритмов численного анализа, для которых априорно оценить время выполнения невозможно.
- Кроме того, используемые на этапах проектирования оценки вычислительной сложности решения подзадач могут иметь приближенный характер, либо количество подзадач может изменяться в ходе вычислений.
- В таких ситуациях может потребоваться перераспределение базовых подзадач между процессорами непосредственно в ходе выполнения параллельной программы, т.е. требуется выполнить динамическую балансировку вычислительной нагрузки.

Схема «менеджер-исполнители» (manager-worker scheme).

- Предполагается, что подзадачи могут возникать и завершаться в ходе вычислений, при этом информационные взаимодействия между подзадачами либо полностью отсутствуют, либо минимальны.
- В системе выделяется отдельный процесс (поток) ОС, играющий роль «менеджера», которому доступна информация о всех имеющихся подзадачах.
- Остальные процессы (потoki ОС) параллельной программы являются исполнителями, которые для получения вычислительной нагрузки обращаются к менеджеру.
- Порождаемые в ходе вычислений новые подзадачи передаются менеджеру и могут быть получены для выполнения при последующих обращениях исполнителей.
- Завершение вычислений происходит в момент, когда исполнители завершили решение всех переданных им задач, а менеджер не имеет каких-либо вычислительных работ для выполнения.
- Фактически, это один из вариантов парадигмы «клиент-сервер» параллельного программирования, когда клиенты – исполнители обращаются к серверу – менеджеру для получения очередной подзадачи.

Перечень контрольных вопросов для проверки этапа распределения подзадач состоит в следующем:

- а) Приводит ли распределение нескольких задач на один процессор к росту дополнительных вычислительных затрат?
- б) Есть ли необходимость динамической балансировки вычислительной нагрузки?
- с) Не является ли процесс – менеджер (поток – менеджер) «узким местом» при использовании схемы «менеджер - исполнитель»?

ПАРАЛЛЕЛЬНЫЕ МЕТОДЫ РЕШЕНИЯ ЗАДАЧ ЛИНЕЙНОЙ АЛГЕБРЫ

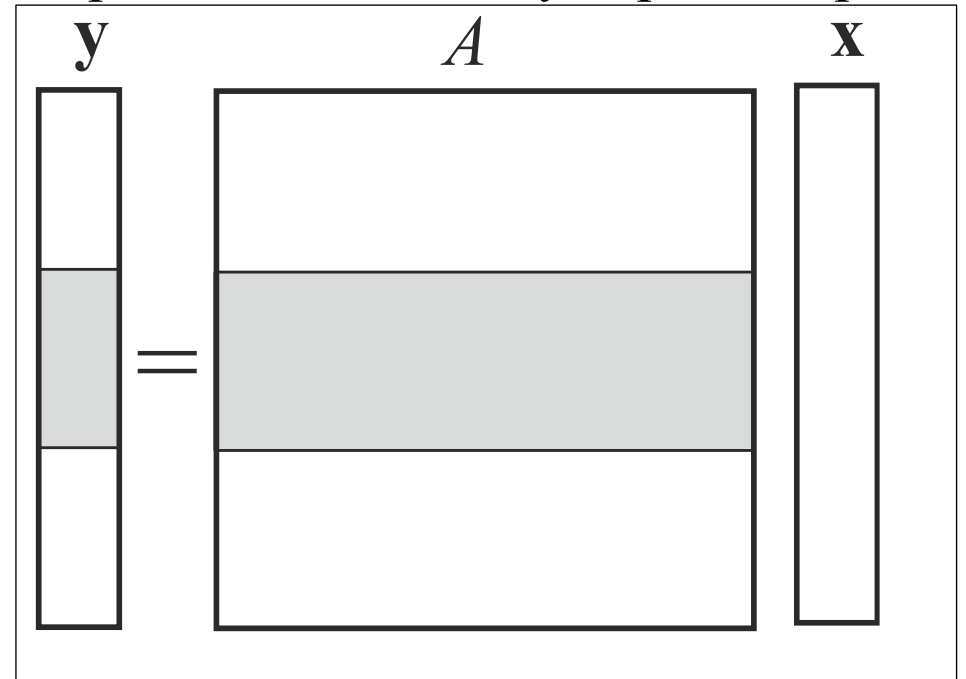
Параллельное умножение матрицы на вектор

$p \in \mathbb{N}$ – число имеющихся в вычислительной системе процессоров

$$\mathbf{y} = \mathbf{A}\mathbf{x}, \quad \mathbf{x} \in \mathbb{R}^N, \quad \mathbf{y} \in \mathbb{R}^M, \quad \mathbf{A} \in \mathbb{R}^{(M,N)}$$

$$y_k = \sum_{j=1}^N a_{kj} x_j, \quad \mathbf{x} = (x_1, x_2, \dots, x_N)^T, \quad \mathbf{y} = (y_1, y_2, \dots, y_M)^T, \quad \mathbf{A} = \{a_{kj}\}, \quad k = \overline{1, M}, \quad j = \overline{1, N}$$

- Каждый элемент вектора $\mathbf{y}$ представляет собой скалярное произведение соответствующей вектор-строки матрицы $\mathbf{A}$ и вектор-столбца $\mathbf{x}$
- $y_k = \mathbf{a}_k^{(r)} \cdot \mathbf{x}, \quad \mathbf{a}_k^{(r)} = (a_{k1}, a_{k2}, \dots, a_{kM})^T$
- Элементы вектора $\mathbf{y}$ вычисляются независимо.
- Если матрица $\mathbf{A}$, вектор $\mathbf{x} \rightarrow \underset{\mathbf{x}}{\text{и}}$, соответственно, вектор $\mathbf{y}$ являются плотно заполненными, то наиболее целесообразно разделение между процессорами фрагментов вектора $\mathbf{y} \rightarrow \underset{\mathbf{y}}{\text{примерно равной}}$ высоты M/p и полос матрицы $\mathbf{A}$ примерно равной высоты M/p .
- Т.к. длительности операций умножения и сложения вещественных чисел фиксированы, то временем выполнения каждой из процессоров своей части вычислительной работы будут примерно одинаковы, и динамической балансировки вычислительной нагрузки не требуется.



Разреженные векторы и матрицы

- Вектор $\mathbf{V} = (V_1, V_2, \dots, V_N)^T \in \mathbb{R}^N$ характеризуется очень большой размерностью $N \gg 1$, но содержит относительно небольшое число $n \ll N$ ненулевых элементов. Его хранят в разреженной форме: $\mathbf{V} = \{\mathbf{v}, \mathbf{i}\}$, где $\mathbf{v} = (v_0, v_1, \dots, v_{n-1})^T \in \mathbb{R}^n$ содержит значения ненулевых элементов вектора $\mathbf{V}$, а $\mathbf{i} = (i_0, i_1, \dots, i_{n-1})^T \in \mathbb{N}^n$, $1 \leq i_0 < i_1 < \dots < i_{n-1} \leq N$ содержит соответствующие им значения индексов.
- Если $\mathbf{U} = (U_1, U_2, \dots, U_N)^T \in \mathbb{R}^N$ – плотно-заполненный вектор, то:

$$\mathbf{U} \cdot \mathbf{V} = \sum_{j=1}^N U_j V_j = \sum_{j=1}^n U_{i_j} v_j, \quad \mathbf{U} \leftarrow \mathbf{U} + \alpha \mathbf{V} \Leftrightarrow U_{i_j} \leftarrow U_{i_j} + \alpha v_j$$

- Операции скалярного умножения плотно заполненного вектора на разреженный и обновления плотно заполненного вектора разреженным выполняются очень быстро
- Пусть вектор $\mathbf{U} \xrightarrow{\mathbf{U}}$ также является N -мерным разреженным вектором, т.е.

$$\mathbf{U} = \{\mathbf{u}, \mathbf{k}\}, \quad \mathbf{u} = (u_0, u_1, \dots, u_{m-1})^T \in \mathbb{R}^m, \quad \mathbf{k} = (k_0, k_1, \dots, k_{m-1})^T \in \mathbb{N}^m, \quad 1 \leq k_0 < k_1 < \dots < k_{m-1} \leq N$$

- Целочисленные векторы $\mathbf{i}$ и $\mathbf{k}$ можно рассматривать как множества целых чисел, и

$$\mathbf{U} \cdot \mathbf{V} = \sum_{j=1}^N U_j V_j = \sum_{k_j = i_l \in \mathbf{k} \cap \mathbf{i}} u_j v_l$$

- Пусть теперь вектор

$$\mathbf{W} = \alpha \mathbf{U} + \beta \mathbf{V} = \{\mathbf{w}, \mathbf{k} \cup \mathbf{i}\}, \mathbf{w} = (w_0, w_1, \dots, w_{|\mathbf{k} \cup \mathbf{i}|-1})^T$$

представляет собой линейную комбинацию разреженных векторов $\mathbf{U}$ и $\mathbf{V}$, причем

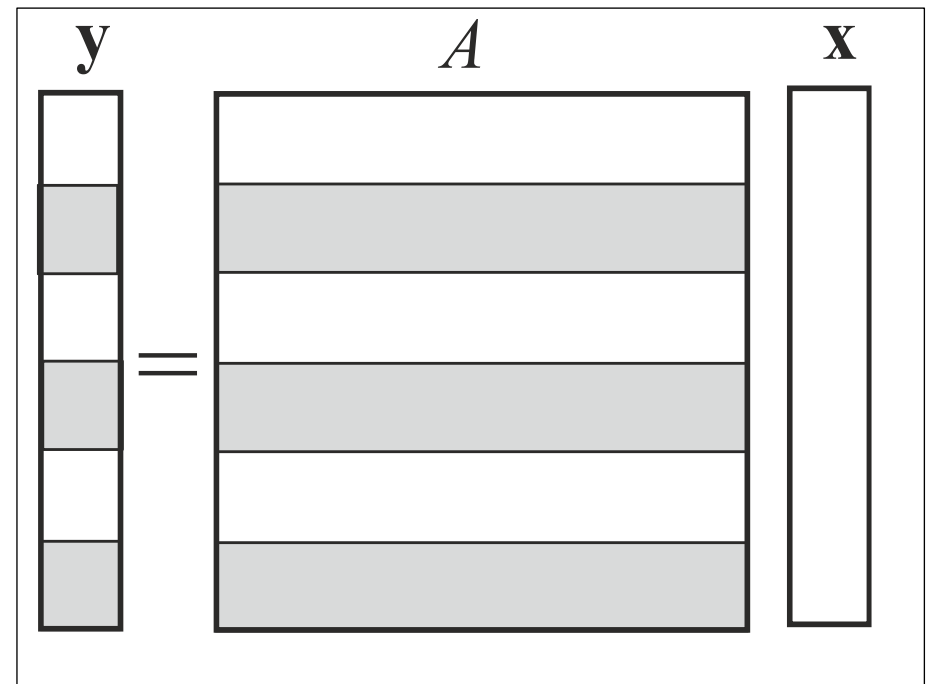
$$w_m = \begin{cases} \alpha u_j + \beta v_l, & m = k_j = j_l \in \mathbf{k} \cap \mathbf{i} \\ \alpha u_j, & m = k_j = j_l \in \mathbf{k} \setminus \mathbf{i} \\ \beta v_l, & m = j_l \in \mathbf{i} \setminus \mathbf{k} \end{cases} \quad m \in \mathbf{k} \cup \mathbf{i}$$

– может потребовать перераспределения оперативной памяти.

Разреженные матрицы

- Если матрица имеет большую размерность $M \times N$, $MN \gg 1$, и содержит значительно меньшее, чем MN , число ненулевых элементов, ее имеет смысл хранить в разреженном формате.
- Если матрица хранится в сжатом по строкам формате, то фактически она представляется в виде массива разреженных вектор-строк (возможно, пустых), в которых индексы соответствуют номерам столбцов, содержащих ненулевые элементы.
- Если матрица хранится в сжатом по столбцам формате, то фактически она представляется в виде массива разреженных вектор-столбцов, в которых индексы соответствуют номерам строк, содержащих ненулевые элементы.

- Координатный формат. Матрица $A = \{a_{ij}\}$, $i=\overline{1, M}$, $j=\overline{1, N}$ является разреженной, и содержит n ненулевых элементов, $n \ll MN$. Тогда $A = \{\mathbf{v}, \mathbf{i}, \mathbf{j}\}$, где $\mathbf{v} = (v_0, v_1, \dots, v_{n-1})^T$ хранит значения ненулевых элементов матрицы A , $\mathbf{i} = (i_0, i_1, \dots, i_{n-1})^T$ хранит их номера строк, а $\mathbf{j} = (j_0, j_1, \dots, j_{n-1})^T$ хранит их номера столбцов, причем при $m \neq l$ $i_m \neq i_l$ либо $j_m \neq j_l$
- Пусть матрица A является разреженной и хранится в сжатом по строкам формате, а вектор $\mathbf{x}$ и, соответственно, вектор $\mathbf{y}$ являются плотно заполненными. Целесообразно разделить вектор $\mathbf{y}$ на некоторое число фрагментов примерно одинаковой высоты, а матрицу A – на такое же число горизонтальных полос соответствующей высоты, так, чтобы число фрагментов/полос было на порядок больше числа доступных процессоров p . Вычисление отдельного фрагмента вектора $\mathbf{y}$ выделяется в отдельную подзадачу. Время выполнения каждой из подзадач априорно непредсказуемо, и целесообразна динамическая балансировка вычислительной нагрузки между процессорами по схеме «менеджер - исполнители».

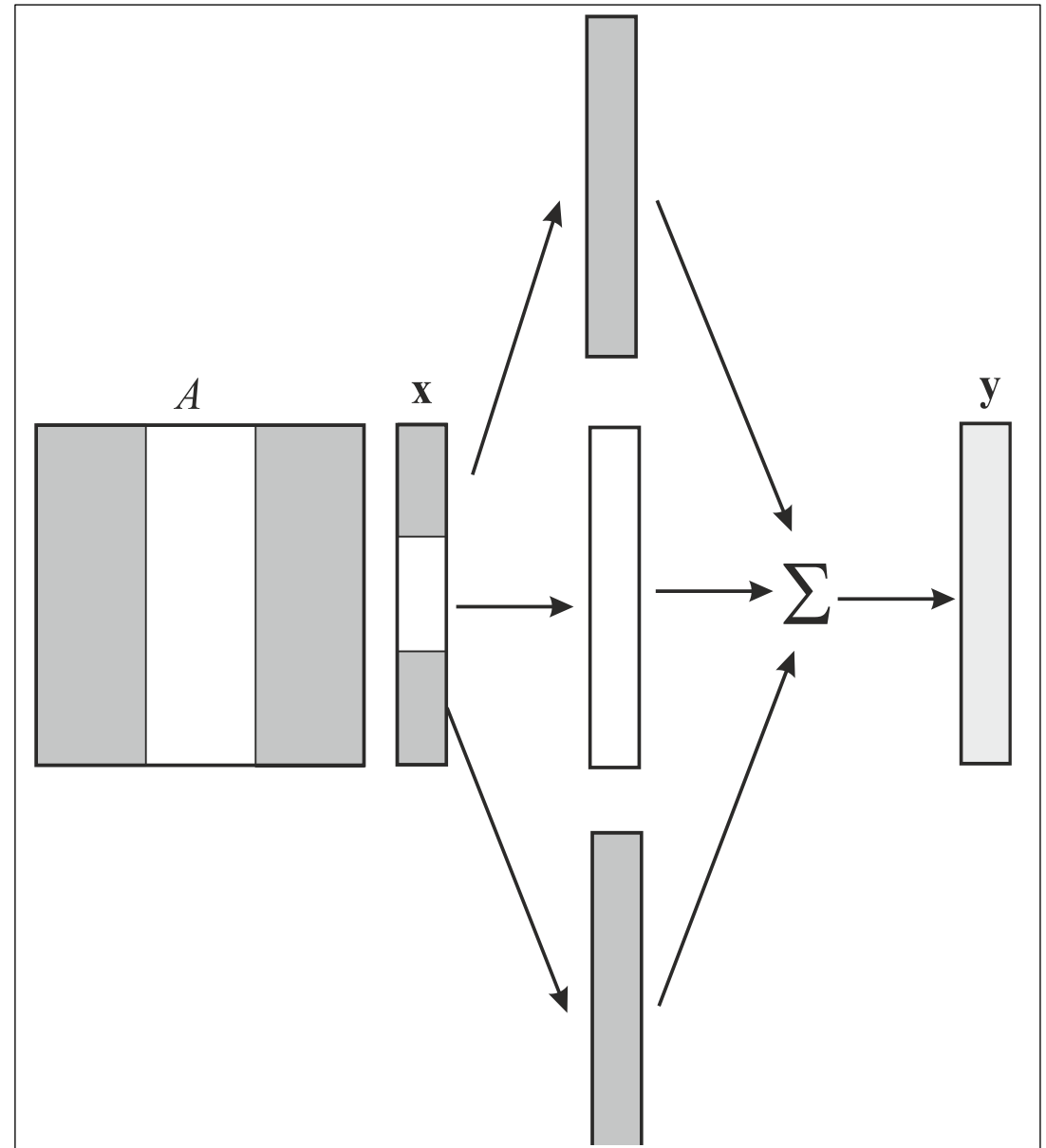


- Если обозначить столбец матрицы A $\mathbf{a}_j^{(c)} = (a_{1j}, a_{2j}, \dots, a_{mj})^T$, то $\mathbf{y} = A\mathbf{x} = \sum_{j=1}^N x_j \mathbf{a}_j^{(c)}$

В последовательном варианте: $\mathbf{y} = 0$; $\mathbf{y} \leftarrow \mathbf{y} + x_j \mathbf{a}_j^{(c)}$, $j = \overline{1, N}$

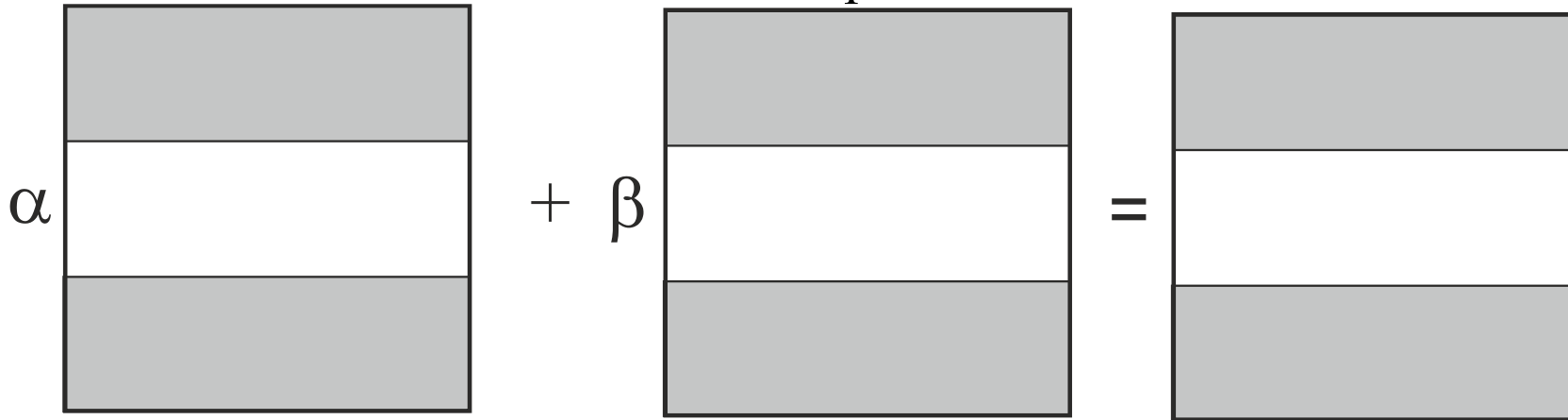
- Пусть матрица A – разреженная очень большой размерности, и хранится в сжатом по столбцам формате, причем в каждом столбце матрицы A хранится примерно одинаковое количество ненулевых элементов. Матрица A разделяется на p вертикальных полос (т.е. групп столбцов) ширины N/p , вектор $\mathbf{x}$ разделяется на p фрагментов высоты N/p без динамической балансировки вычислительной нагрузки. Требуется дополнительный объем оперативной памяти для размещения $(p-1)$ векторов, размерность которых совпадает с размерностью вектора $\mathbf{y}$

- Динамическая балансировка нагрузки возможна, но может приводит к значительно большим дополнительным затратам оперативной памяти

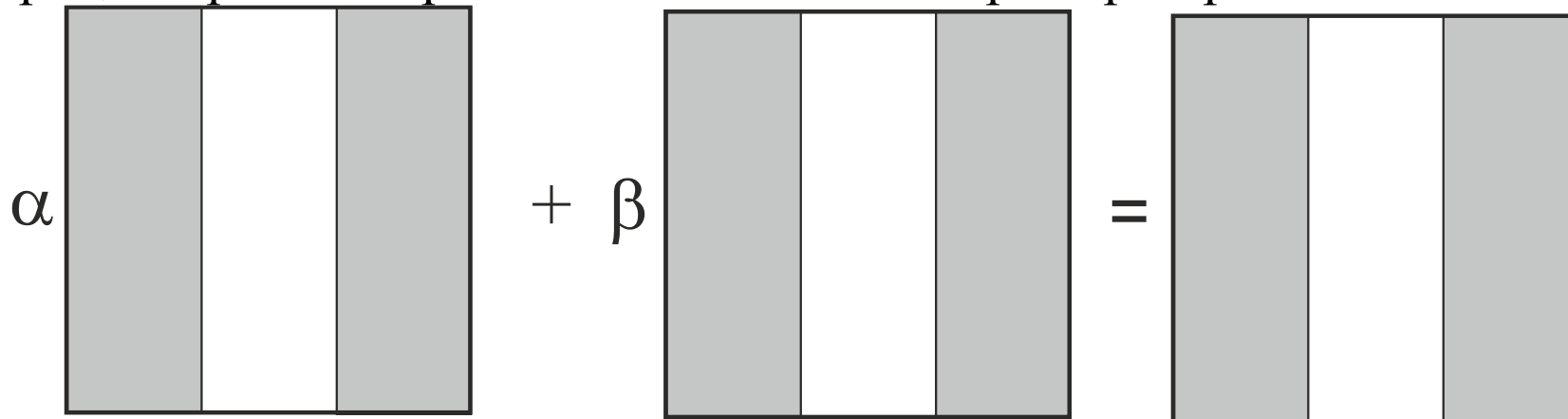


Линейная комбинация матриц $C = \alpha A + \beta B$

- Если матрицы являются плотно заполненными, то это операция аналогична операции над достаточно длинными векторами.



- Если элементы матриц упорядочены по строкам, то матрицы разбиваются по числу процессоров на горизонтальные полосы примерно равной высоты.



- Если элементы матриц упорядочены по столбцам, то матрицы разбиваются по числу процессоров на вертикальные полосы примерно равной высоты.

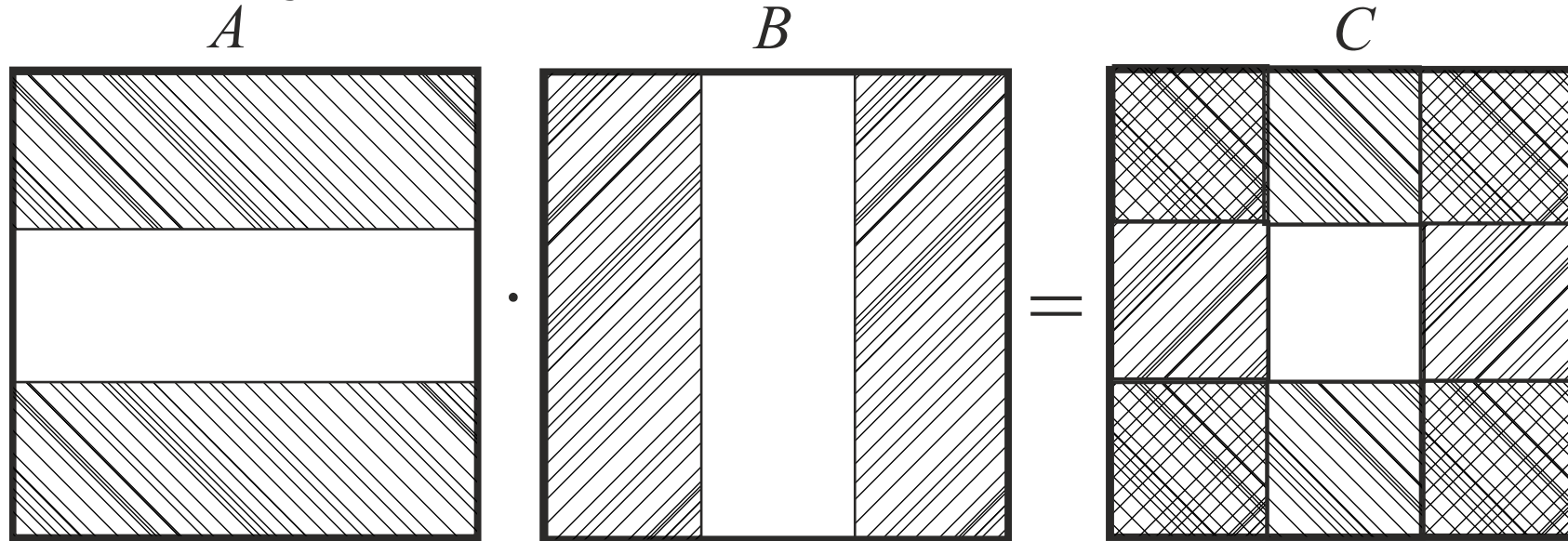
- Для плотно заполненных матриц не требуется динамической балансировки вычислительной нагрузки.
- Если матрицы являются разреженными и хранятся в форме, сжатой по строкам, то матрицы разбиваются на достаточно большое число узких горизонтальных полос, на порядок больше, чем число процессоров.
- Если они хранятся в форме, сжатой по столбцам, то матрицы разбиваются на достаточно большое число узких вертикальных полос, на порядок больше, чем число процессоров.
- Обработка отдельной полосы выделяется в подзадачу. Используется динамическая балансировка вычислительной нагрузки.

Перемножение матриц

$$C = AB, \text{ где } A = \{a_{kj}\}, \quad k = \overline{1, N_1}, \quad j = \overline{1, N_2}; \quad B = \{b_{kj}\}, \quad k = \overline{1, N_2}, \quad j = \overline{1, N_3},$$

$$C = \{c_{kj}\}, \quad k = \overline{1, N_1}, \quad j = \overline{1, N_3}; \quad c_{kj} = \sum_{i=1}^{N_2} a_{ki} b_{ij}, \quad k = \overline{1, N_1}, \quad j = \overline{1, N_3}$$

- Все элементы матрицы C могут быть вычислены независимо.
- Пусть $N_1 = N_2 = N_3 = N$. Для плотно заполненных матриц:



Первый сомножитель-матрицу A – можно разбить на p_1 горизонтальных полос примерно равно высоты, в второй сомножитель-матрицу B – можно разбить на p_2 вертикальных полос равной ширины. Тогда вычисление $p = p_1 p_2$ блоков матрицы C может быть выполнено параллельно на p процессорах.

- Если используется кластерная система с распределенной памятью, то на некоторый узел потребуется выполнить пересылку $N^2 \left(\frac{1}{p_1} + \frac{1}{p_2} \right)$ элементов исходных матриц A и B .
- При этом на отдельном узле потребуется выполнить $\frac{(2N-1)N^2}{p_1 p_2}$ операций умножения и сложения
- При $N \gg 1$ и конечных p_1, p_2 число вычислительных операций будет значительно, в $\frac{2N-1}{p_1 + p_2}$ раз превосходить число пересылаемых элементов.
- Поскольку все процессоры выполняют одинаковое количество вычислительной работы, то динамическая балансировка вычислительной нагрузки на процессоры здесь не требуется.
- В пределах отдельно взятого процесса или потока операция умножения плотно заполненных матриц может быть на порядок ускорена за счет оптимального использования кэш-памяти отдельно взятого процессора.

Оптимизация кэш-памяти

$$C = AB, \quad A = \{a_{kj}\}, \quad k = \overline{1, N_1}, \quad j = \overline{1, N_2};$$

$$B = \{b_{kj}\}, \quad k = \overline{1, N_2}, \quad j = \overline{1, N_3}, \quad C = \{c_{kj}\}, \quad k = \overline{1, N_1}, \quad j = \overline{1, N_3}.$$

- Принимая $N_1 = n_1 n$, $N_2 = n_2 n$, $N_3 = n_3 n$, разобьем матрицы A , B и C на блоки размерности $n \times n$

$$A_{ik} = \{a_{jl}\}, \quad j = \overline{(i-1)n+1, in}, \quad l = \overline{(k-1)n+1, kn}, \quad i = \overline{1, n_1}, \quad k = \overline{1, n_2}$$

$$B_{ik} = \{b_{jl}\}, \quad j = \overline{(i-1)n+1, in}, \quad l = \overline{(k-1)n+1, kn}, \quad i = \overline{1, n_2}, \quad k = \overline{1, n_3}$$

$$C_{ik} = \{c_{jl}\}, \quad j = \overline{(i-1)n+1, in}, \quad l = \overline{(k-1)n+1, kn}, \quad i = \overline{1, n_1}, \quad k = \overline{1, n_3}$$

$$A = \begin{bmatrix} A_{11} & A_{12} & \dots & A_{1n_2} \\ A_{21} & A_{22} & \dots & A_{2n_2} \\ \dots & \dots & \dots & \dots \\ A_{n_1 1} & A_{n_1 2} & \dots & A_{n_1 n_2} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} & \dots & B_{1n_3} \\ B_{21} & B_{22} & \dots & B_{2n_3} \\ \dots & \dots & \dots & \dots \\ B_{n_2 1} & B_{n_2 2} & \dots & B_{n_2 n_3} \end{bmatrix}, \quad C = \begin{bmatrix} C_{11} & C_{12} & \dots & C_{1n_3} \\ C_{21} & C_{22} & \dots & C_{2n_3} \\ \dots & \dots & \dots & \dots \\ C_{n_1 1} & C_{n_1 2} & \dots & C_{n_1 n_3} \end{bmatrix}$$

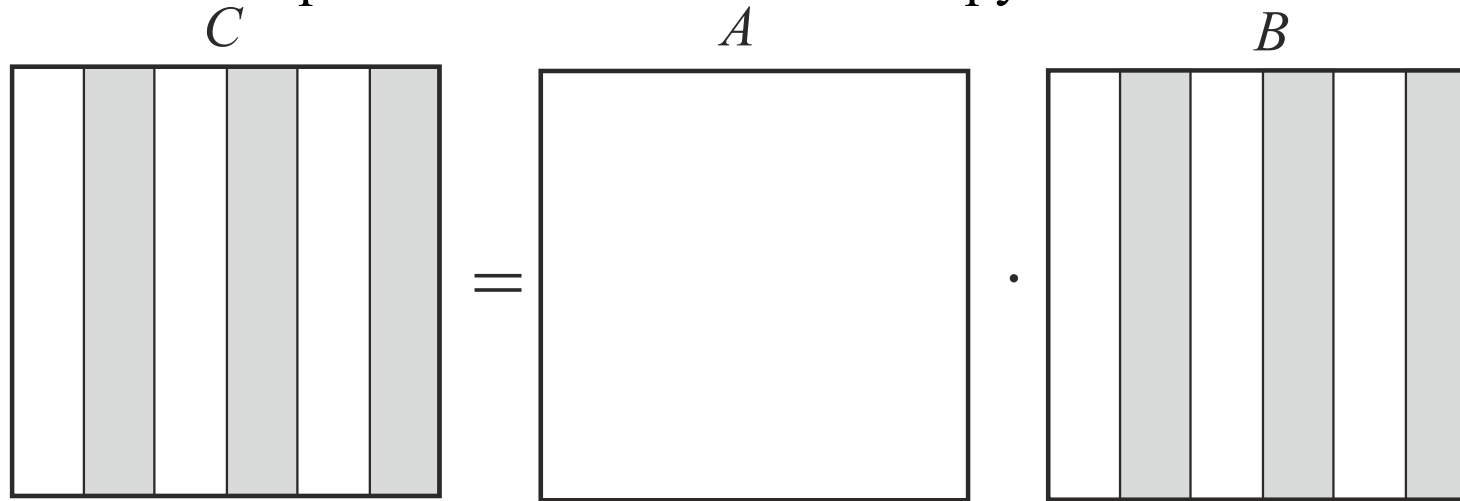
- Каждый блок рассматриваем как матрицу размерности $n \times n$. В результате

$$C_{ik} = \sum_{j=1}^{n_2} A_{ij} B_{jk}$$

- При добавлении к матрице C_{ik} очередного слагаемого $A_{ij}B_{jk}$ матрица C_{ik} находится в кэш-памяти, а матрицы A_{ij} и B_{jk} целиком заносятся в кэш-память.
- Это реализуемо, если не менее чем $3n^2$ элементов матриц (A , B и C) полностью умещаются в кэш-памяти процессора.
- Т.е. при этом в кэш-память заносится $2n^2$ элементов в матрицу A_{ij} и B_{jk} , но в пределах кэш-памяти выполняется $(2n-1)n^2$ операций умножения и сложения, связанных с перемножением данных матриц (т.е. существенно большее число). Т.к. доступ к кэш-памяти на порядок быстрее, чем к оперативной памяти, это на порядок ускоряет вычисления.
- Различные реализации библиотек BLAS поддержки высокопроизводительных вычислений используют оптимизацию кэш-памяти, специфичные ассемблерные команды и распараллеливание на основе многопоточности для ускорения матричных вычислений.

Перемножение разреженных матриц

- A , B и $C = AB$ – разреженные матрицы большой размерности в сжатом по столбцам формате
- Умножение матрицы A на разреженный вектор-столбец сводится к линейной комбинации подмножества разреженных столбцов матрицы A , соответствующих ненулевым компонентам вектора.
- Целесообразно разделение матриц B и C на достаточно большое число узких вертикальных полос, на порядок большее, чем число процессоров p . Вычисление текущей полосы матрицы C выделяется в отдельную подзадачу. Используется динамическая балансировка вычислительной нагрузки.



- При помощи преобразования транспонирования $C = AB \rightarrow C^T = B^T A^T$ умножение сжатых по строкам разреженных матриц может быть сведено к умножению сжатых по столбцам разреженных матриц.

Параллельные методы численного решения СЛАУ

$$A\mathbf{x} = \mathbf{b}, A = \{a_{ij}\} \in \mathbb{R}^{(N,N)}, i, j = \overline{1, N}, N \gg 1; \mathbf{b} = (b_1, b_2, \dots, b_N)^T \in \mathbb{R}^N, \mathbf{x} = (x_1, x_2, \dots, x_N)^T \in \mathbb{R}^N$$

- Пусть невырожденная квадратная матрица A является плотно заполненной.

$$\begin{array}{cccccc} a_{11}x_1 & + a_{12}x_2 & + a_{13}x_3 + \dots & + a_{1,N-1}x_{N-1} & + a_{1,N}x_N = b_1 \\ a_{21}x_1 & + a_{22}x_2 & + a_{23}x_3 + \dots & + a_{2,N-1}x_{N-1} & + a_{2,N}x_N = b_2 \\ a_{31}x_1 & + a_{32}x_2 & + a_{33}x_3 + \dots & + a_{3,N-1}x_{N-1} & + a_{3,N}x_N = b_3 \\ \hdashline a_{N-1,1}x_1 & + a_{N-1,2}x_2 & + a_{N-1,3}x_3 + \dots & + a_{N-1,N-1}x_{N-1} & + a_{N-1,N}x_N = b_{N-1} \\ a_{N,1}x_1 & + a_{N,2}x_2 & + a_{N,3}x_3 + \dots & + a_{N,N-1}x_{N-1} & + a_{N,N}x_N = b_N \end{array}$$

- **Прямой метод Гаусса.** С точностью до перестановки строк и столбцов, исключаем субдиагональные элементы в первом столбце

$$\begin{aligned}
x_1 + a_{12}^{(1)} x_2 + a_{13}^{(1)} x_3 + \dots + a_{1,N-1}^{(1)} x_{N-1} + a_{1,N}^{(1)} x_N &= b_1^{(1)} \\
a_{22}^{(1)} x_2 + a_{23}^{(1)} x_3 + \dots + a_{2,N-1}^{(1)} x_{N-1} + a_{2,N}^{(1)} x_N &= b_2^{(1)} \\
a_{32}^{(1)} x_2 + a_{33}^{(1)} x_3 + \dots + a_{3,N-1}^{(1)} x_{N-1} + a_{3,N}^{(1)} x_N &= b_3^{(1)} \\
\vdots & \\
a_{N-1,2}^{(1)} x_2 + a_{N-1,3}^{(1)} x_3 + \dots + a_{N-1,N-1}^{(1)} x_{N-1} + a_{N-1,N}^{(1)} x_N &= b_{N-1}^{(1)} \\
a_{N,2}^{(1)} x_2 + a_{N,3}^{(1)} x_3 + \dots + a_{N,N-1}^{(1)} x_{N-1} + a_{N,N}^{(1)} x_N &= b_N^{(1)}
\end{aligned}$$

$$a_{1j}^{(1)} = a_{1j} / a_{11}, j = \overline{2, N}, b_1^{(1)} = b_1 / a_{11},$$

$$a_{ij}^{(1)} = a_{ij} - a_{i1} a_{1j}^{(1)}, b_i^{(1)} = b_i - a_{i1} b_1^{(1)}, i, j = \overline{2, N}$$

- Далее рекурсивно применяем этот алгоритм к оставшейся подсистеме относительно $x_2, x_3, \dots, x_N$ и т.д. Матрица системы преобразуется к верхней треугольной

$$x_1 + a_{12}^{(1)} x_2 + a_{13}^{(1)} x_3 + \dots + a_{1,N-1}^{(1)} x_{N-1} + a_{1,N}^{(1)} x_N = b_1^{(1)}$$

$$x_2 + a_{23}^{(2)} x_3 + \dots + a_{2,N-1}^{(2)} x_{N-1} + a_{2,N}^{(2)} x_N = b_2^{(2)}$$

$$x_3 + \dots + a_{3,N-1}^{(3)} x_{N-1} + a_{3,N}^{(3)} x_N = b_3^{(3)}$$

.....

$$x_{N-1} + a_{N-1,N}^{(N-1)} x_N = b_{N-1}^{(N-1)}$$

$$x_N = b_N^{(N)}$$

$$A^{(0)} = A, \mathbf{b}^{(0)} = \mathbf{b}, A^{(k)} = \{a_{ij}^{(k)}\}, i, j = \overline{1, N}, \mathbf{b}^{(k)} = (b_1^{(k)}, b_2^{(k)}, \dots, b_{N-1}^{(k)})^T$$

$$a_{kj}^{(k)} = a_{kj}^{(k-1)} / a_{kk}^{(k-1)}, j = \overline{k+1, N}, b_k^{(k)} = b_k^{(k-1)} / a_{kk}^{(k-1)},$$

$$a_{ij}^{(k)} = a_{ij}^{(k-1)} - a_{ik}^{(k-1)} a_{kj}^{(k)}, b_i^{(k)} = b_i^{(k-1)} - a_{ik}^{(k-1)} b_k^{(k)}, i, j = \overline{k+1, N}, k = \overline{1, N-1}$$

- Обратный ход метода Гаусса

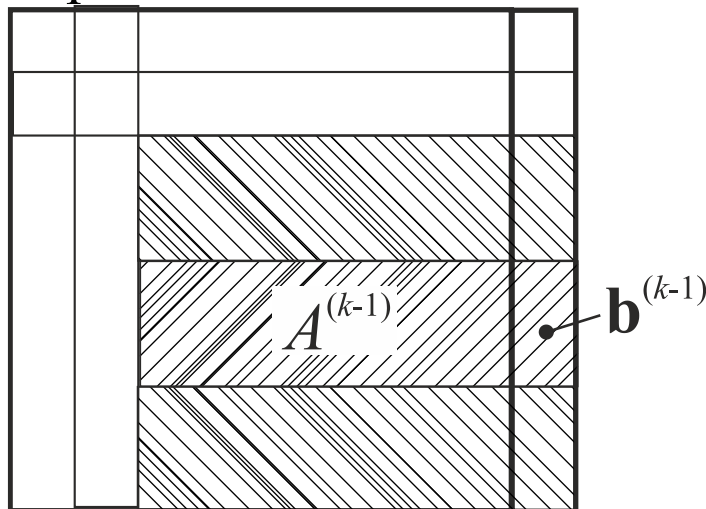
$$x_N = b_N, x_k = b_k - \sum_{j=k+1}^N a_{kj}^{(k-1)} x_j, k = N-1, N-2, \dots, 1$$

- Прямой метод фактически выполняет факторизацию матрицы системы

$$A = LU$$

где L – нижняя треугольная матрица с единицей на главной диагонали, U – верхняя треугольная матрица.

- Пусть $[A^{(k-1)}, \mathbf{b}^{(k-1)}]$ – расширенная матрица преобразуемой системы. На каждом шаге k её преобразование в расширенную матрицу $[A^{(k)}, \mathbf{b}^{(k)}]$ предусматривает преобразование элементов, лежащих ниже k -ой строки и правее k -ого столбца во фрагменте расширенной матрицы.



- При этом строки рассматриваемого фрагмента преобразуются независимо друг от друга. Следовательно, указанный фрагмент целесообразно разделить на горизонтальные полосы примерно равной ширины, и обрабатывать каждую полосу в отдельном процессоре.
- Подобный метод распараллеливания (дополненный использованием блочных алгоритмов работы с матрицами для улучшения использования кэш-памяти) используется в LAPACK
- Формулы для обратного хода метода Гаусса содержат достаточно длинные суммы и также могут быть распараллелены (но менее эффективно)

- **Итерационные методы** решения систем линейных уравнений основаны на их приведении к виду

$$\mathbf{x} = A\mathbf{x} + \mathbf{b}$$

и далее на использовании итерационного процесса

$$\mathbf{x}_0 = \mathbf{b}, \mathbf{x}_{k+1} = A\mathbf{x}_k + \mathbf{b}, k = 0, 1, 2, \dots$$

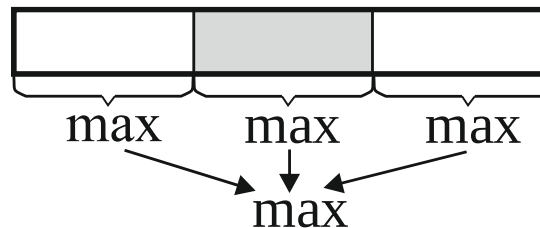
который гарантированно сходится при выполнении условия

$$\|A\| = \left[\sum_{i=1}^N \sum_{j=1}^N |a_{ij}|^2 \right]^{1/2} < 1$$

- Распараллеливание итерационных методов решения систем линейных алгебраических уравнений фактически сводится к распараллеливанию операций умножения матрицы на вектор и суммирования векторов.

ПАРАЛЛЕЛЬНЫЕ МЕТОДЫ СОРТИРОВКИ

- **Параллельный алгоритм поиска максимума (минимума)** в массиве аналогичен параллельному алгоритму суммирования элементов массива. Массив разбивается на примерно равные части, и в каждой из них ищется максимум либо минимум. После этого ищется максимум либо минимум среди найденных значений. При этом может потребоваться динамическая балансировка вычислительной нагрузки, т.к. в зависимости от условия либо выполняется, либо не выполняется одна операция присваивания.



Четно-нечетная перестановка

- Распараллеливаемая модификацию алгоритма пузырьковой сортировки
- Пусть сортируемый массив суть

$$A = \{a_0, a_1, a_2, \dots, a_{N-2}, a_{N-1}\}$$

и итерации алгоритма нумеруются значениями индекса $0, 1, 2, \dots, N-1$.

- На всех четных итерациях упорядочиваются элементы в парах

$$(a_0, a_1), (a_2, a_3), (a_4, a_5), (a_6, a_7), \dots$$

т.е. в парах, где левый элемент имеет четный номер.

На всех нечетных итерациях упорядочиваются элементы в парах

$$(a_1, a_2), (a_3, a_4), (a_5, a_6), (a_7, a_8), \dots$$

т.е. в парах, где левый элемент имеет нечетный индекс.

- После выполнения инструкции с номером $N-1$, т.е. после выполнения общего числа итераций, равного N , массив A оказывается упорядоченным.
- Распараллеливание алгоритма «четно-нечетной перестановки» основано на том, что на любой стадии сравнения элементов и возможные их перестановки в любых различных парах можно выполнить независимо, т.е. данные операции можно выполнить одновременно.

Сортировка слиянием двух упорядоченных массивов

$A = \{a_0, a_1, a_2, \dots, a_{N_a-2}, a_{N_a-1}\}$, $a_{j+1} \geq a_j$ и $B = \{b_0, b_1, b_2, \dots, b_{N_b-2}, b_{N_b-1}\}$, $b_{j+1} \geq b_j$

и помещение результата в новый упорядоченный массив

$C = \{c_0, c_1, c_2, \dots, c_{N-2}, c_{N-1}\}$, $N = N_a + N_b$, $c_{j+1} \geq c_j$

Если $a_{N_a} \leq b_0$, то сначала в массив C помещаем все элементы массива A , а сразу после них помещаем все элементы массива B , и заканчиваем работу алгоритма.

Если $b_{N_b} < a_0$, то сначала в массив C помещаем все элементы массива B , сразу после них помещаем все элементы массива A , и завершаем работу алгоритма.

В противном случае, полагаем, что номера текущих элементов входных массивов A , B и выходного массива C равны нулю. Пока не будет исчерпан один из входных массивов (A или B), выполняем следующие действия:

Выбираем минимальный из текущих элементов в массивах A и B и помещаем его в текущую позицию в массив C . Увеличиваем на 1 номер текущего элемента в выходном массиве C и в том из выходных массивов (A или B), из которого был выбран минимальный элемент.

Как только будет исчерпан один из входящих массивов (A или B), то, если не исчерпан другой входящий массив, начиная с текущей позиции в неисчерпанном входном массиве и с текущей позиции в выходном массиве C , последовательно записываем элементы неисчерпанного входного массива в выходной массив C .

После этого завершаем работу алгоритма. Трудоемкость: $\underline{\underline{O(N)}}$.

Блочный аналог четно-нечетной перестановки.

$p \in \mathbb{N}$ — число процессов, $p \ll N$, $N = 2pn$, и массив A разбит на блоки $A_0, A_1, \dots, A_{2p-1}$ равной длины n : $A = \{A_0, A_1, \dots, A_{2p-1}\}$, $A_k = \{a_{nk}, a_{nk+1}, \dots, a_{n(k+1)-1}\}$

Если $\forall a_{k_1} \in A_k, a_{j_1} \in A_j \quad a_{k_1} \geq a_{j_1}$, то это обозначают $A_k \geq A_j$ (9)

$a_{k1} > a_{j1} \dots \dots \dots A_k > A_j$ $a_{k1} \leq a_{j1} \dots \dots \dots A_k \leq A_j$ (10)
 $a_{k1} < a_{j1} \dots \dots \dots A_k < A_j$

Пусть в пределах каждого из блоков $A_0, A_1, \dots, A_{2p-1}$ элементы исходного массива уже упорядочены, т. е. $\forall a_j, a_{j+1} \in A_k \quad a_{j+1} \geq a_j$. В таком случае, проверка условия (9) сводится к проверке условия $a_{nk} \geq a_{n(j+1)-1}$, а проверка условия (10) сводится к проверке условия $a_{n(k+1)-1} \leq a_{nj}$.

Пусть массив A состоит из блоков, в каждом из которых элементы упорядочены. Итерации алгоритма нумеруются посредством значений $0, 1, 2, \dots, 2p-2, 2p-1$ некоторого индекса.

На всех четных итерациях в парах блоков $(A_0, A_1), (A_2, A_3), (A_4, A_5), (A_6, A_7), \dots$ проверяется выполнение условия $A_{2j} \leq A_{2j+1}$, и, если оно не выполняется, то выполняется упорядочивание элементов в соответствующей паре блоков.

На всех нечетных итерациях в парах блоков $(A_1, A_2), (A_3, A_4), (A_5, A_6), (A_7, A_8), \dots$ проверяется выполнение условия $A_{2j-1} \leq A_{2j}$, и, если оно не выполняется, то выполняется упорядочивание элементов массива в соответствующей паре блоков.

Упорядочивание элементов в парах блоков выполняется алгоритмом слияния упорядоченных массивов, соответствующих данным блокам. При этом, если $A_{2j-1} \geq A_j$, то эти блоки просто размениваются местами.

Данные операции, выполняемые над парами блоков, часто называются так: «сравнить и разделить».

По аналогии с обычным вариантом четно-нечетной перестановки, можно утверждать, что после выполнения итерации с номером $2p-1$, т.е. общего числа $2p$ итераций, исходный массив A станет упорядоченным.

Последовательный вариант данного алгоритма характеризуется временем выполнения, не превышающем следующей оценки: $\sim 2p \cdot p \cdot 2n = 2pN$, что существенно лучше, чем время выполнения последовательного варианта «четно-нечетной» перестановки, равного $\sim N^2$.

Распараллеливание блочного варианта «четно-нечетной перестановки» основано на том, что на каждой итерации упорядочивание элементов в любых различных парах блоков можно выполнить независимо, т.е. эти операции можно выполнить одновременно. Параллельный вариант блочного алгоритма «четно-нечетной перестановки» характеризуется временем выполнения, не большим асимптотической оценки: $\sim 2N$, что существенно лучше, чем характерное время выполнения параллельного варианта алгоритма «четно-нечетной» перестановки, равное $\sim N^2 / p$.

На кластерных системах: пересылка соседних блоков между соседними узлами.

Быстрая сортировка Хоара.

На первой итерации метода реализуется деление исходного набора данных на первые две части. Выбирается некоторый ведущий элемент и все значения массива, меньше ведущего элемента, переносятся в первый блок, а все остальные значения переносятся во второй блок некоторого нового массива, размер которого совпадает с размером исходного массива.

На второй итерации быстрой сортировки данные правила рекурсивно применяются для обоих отсортированных блоков и т.д.

При оптимальном выборе ведущих элементов, когда деление каждого блока выполняется на примерно равные части, после выполнения примерно $\log_2(N)$ итераций исходный массив оказывается упорядоченным, и трудоемкость данного алгоритма оценивается величиной $N \log_2(N)$. Такой же порядок имеет трудоемкость быстрой сортировки «в среднем».

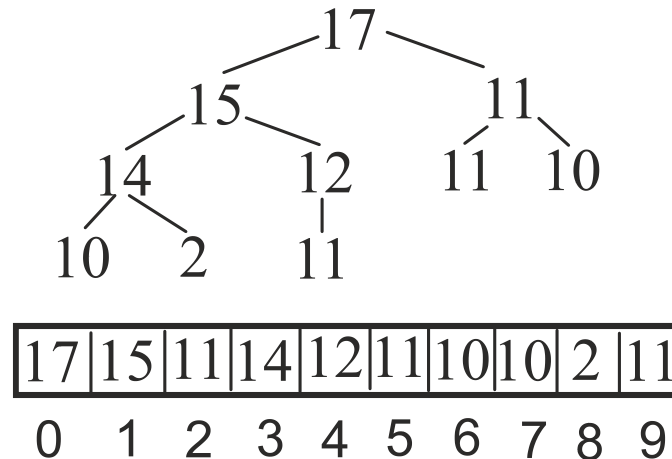
Однако в особо неблагоприятных случаях, когда исходный массив все время делится на две части, в одной из которых содержится лишь один элемент, алгоритм деградирует, а его трудоемкость оценивается как $\sim N^2$, т.е. как и в случае простой сортировки.

Тем не менее, поскольку алгоритм быстрой сортировки рекурсивно дважды независимо вызывает сам себя, он может быть легко распараллелен, и представляет собой пример алгоритма с явно выраженным рекурсивным параллелизмом.

Пирамидальная сортировка

Трудоемкость: $\sim N \log_2(N)$.

Используется частично упорядоченное бинарное дерево (неубывающая пирамида), которое проецируется в массив.



- На нижнем уровне, где некоторые листья могут отсутствовать, требуется, чтобы все отсутствующие листья в принципе могли располагаться лишь справа от присутствующих листьев нижнего уровня, т.е. на нижнем уровне присутствующие листья располагаются как можно левее;

- Дерево частично упорядочено, т.е. значение, которое хранится в узле (вершине) v , не может быть меньше, чем значение, хранимое в любом из его «сыновей».

При проецировании в массив данной структуры данных сначала в массиве располагают вершину дерева, затем — узлы на первом уровне, затем — узлы на втором уровне, затем — узлы третьего уровня и т.д.

Пусть элементы массива нумеруются с нуля. Если i -номер узла в массиве, а k_l и k_r -номера в массиве его «левого» и «правого» сыновей, то

$$\begin{aligned} k_l + 1 &= 2(i + 1), \quad k_r + 1 = 2(i + 1) + 1, \text{ или} \\ k_l &= 2i + 1, \quad k_r = 2i + 2 \end{aligned} \quad (11)$$

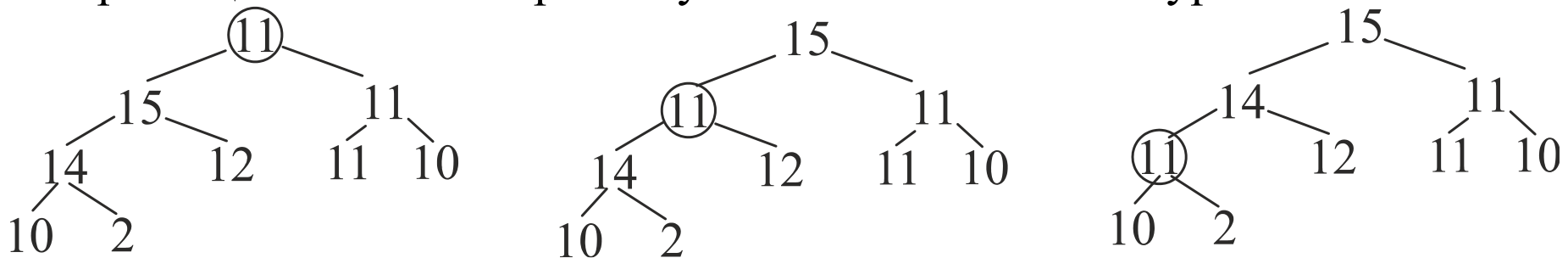
Если k – номер в массиве некоторого узла, а i – номер в массиве его «отца», то

$$i = \left\lfloor \frac{(k-1)}{2} \right\rfloor = \left\lfloor \frac{(k+1)}{2} \right\rfloor - 1 \quad (12)$$

Пусть N – число элементов в частично упорядоченном сбалансированном бинарном дереве. Число уровней без единицы в дереве, т.е. его высота, суть $i = \left\lfloor \frac{(k-1)}{2} \right\rfloor = \left\lfloor \frac{(k+1)}{2} \right\rfloor - 1$.

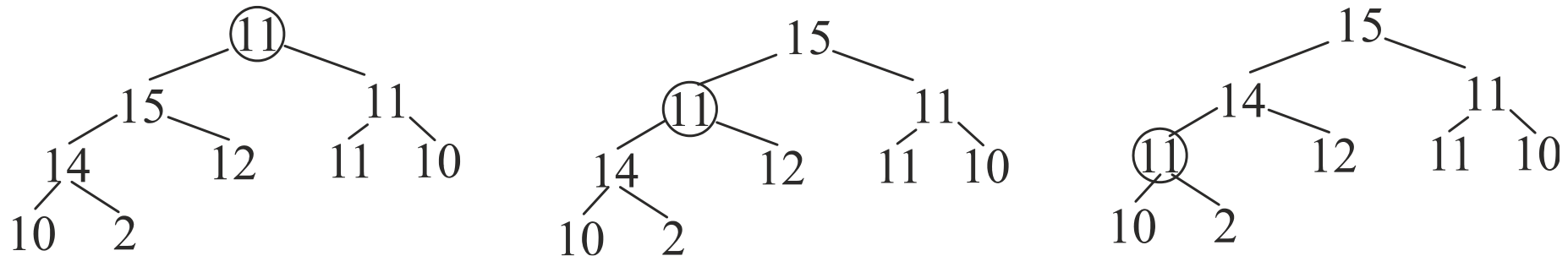
Операция «извлечь максимальный элемент и удалить его из дерева» - DELETEMAX распадается на две фазы.

Первая фаза DELETEMAX: выдается значение в корне дерева, а затем в корень дерева перемещается самый правый узел на самом нижнем уровне.



Вторая фаза DELETEMAX: к корню дерева применяется операция «протолкнуть вниз» - PUSHDOWN для обеспечения частичной упорядоченности бинарного сбалансированного дерева.

Операция PUSHDOWN предполагает, что условия частичной упорядоченности выполняются для поддеревьев с корнями в левом и правом сыновьях корня данного дерева, но могут нарушаться в корне данного дерева из-за того, что он хранит меньшее значение, чем его сыновья. Операция PUSHDOWN поэтапно продвигает корень дерева вниз, обменивая его местами с тем из его «сыновей», который содержит большее значение, вплоть до момента, когда будет достигнута частичная упорядоченность, т.е. когда значение в данном узле станет меньше, чем значение, хранимое в его «сыновьях».

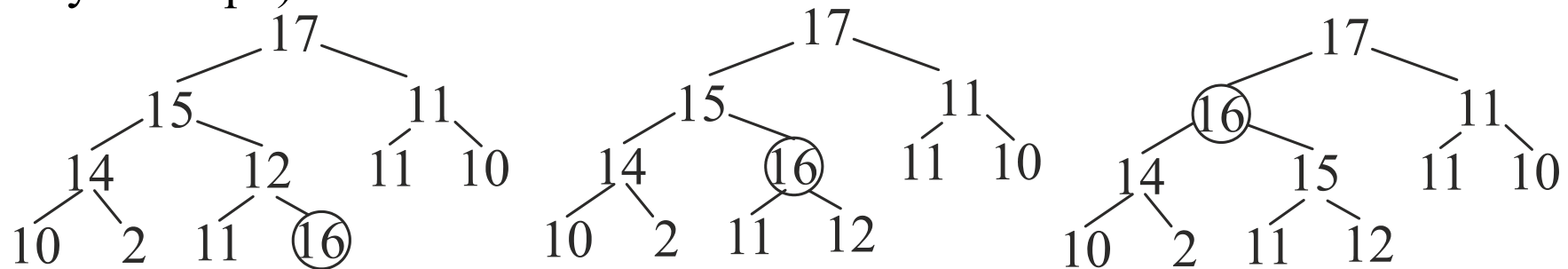


Трудоемкость операции DELETEMAX совпадает с трудоемкостью операции PUSHDOWN, определяется высотой дерева и асимптотически оценивается величиной $i = \lfloor \frac{(k-1)}{2} \rfloor = \lfloor \frac{(k+1)}{2} \rfloor - 1$. Если дерево состоит лишь из корня, то операция PUSHDOWN ничего не выполняет.

Вставка нового элемента в бинарное частично упорядоченное дерево реализуется стандартной операцией INSERT, которая также распадается на две фазы.

На первой фазе новый элемент помещается в самую левую свободную позицию на самом нижнем уровне. Если этот уровень заполнен, то начинается новый уровень.

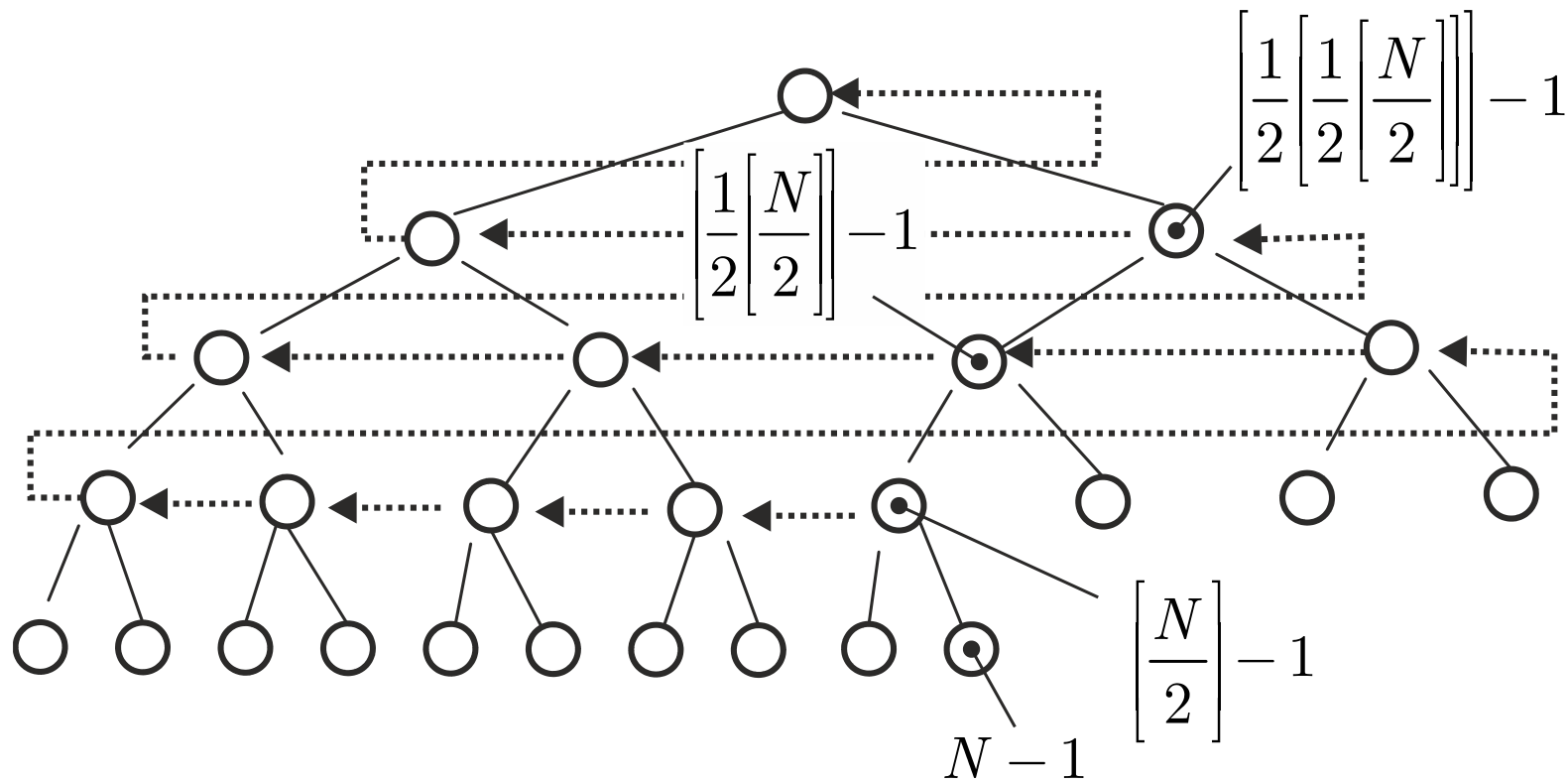
На второй фазе к вставленному элементу применяется операция PUSHUP (продвинуть вверх).



Если значение в узле, к которому применяется операция PUSHUP, превышает значение, хранимое в его родителе, то операция PUSHUP поочередно продвигает узел вверх, обменивая его местами с его родителем, вплоть до момента, когда будет достигнута частичная упорядоченность, т.е. когда значение, хранимое в родителе данного узла, станет не меньше, чем значение, хранимое в данном узле. Трудоемкость операции PUSHUP определяется номером уровня, на котором расположен узел, к которому она применяется, т.е. глубиной узла. Трудоемкость операции INSERT определяется высотой дерева, т.е. асимптотически определяется величиной $\log_2(N)$.

Если происходит уменьшение значения, хранимого в узле, то для восстановления частичной упорядоченности к корню поддерева, совпадающему с текущим узлом, применяется операция PUSHDOWN. Если происходит увеличение значения, хранимого в текущем узле, то для восстановления частичной упорядоченности к данному узлу применяется операция PUSHUP. И в том, и в другом случае трудоемкость не превышает $i = \lceil \frac{(k-1)}{2} \rceil = \lceil \frac{(k+1)}{2} \rceil - 1$.

При создании частично упорядоченного дерева можно сначала спроецировать дерево в массив, а затем добиться его частичной упорядоченности. Нижний самый правый узел, имеющий хотя бы одного «сына», характеризуется номером $\left\lfloor \frac{N}{2} \right\rfloor - 1$ в массиве. Двигаясь от узла с номером в массиве $\left\lfloor \frac{N}{2} \right\rfloor - 1$ к корню дерева (узлу с номером 0 в массиве), т.е. двигаясь в дереве справа налево с переходом на уровень вверх, поочередно применяем операцию PUSHROWN к поддеревьям с корнем в текущем узле.



При создании бинарного сбалансированного частично упорядоченного дерева:
 для узлов с номерами от $\approx N/2$ до $\approx N/4$ операция PUSHDOWN обрабатывает
 деревья высоты 1;

Для узлов с номерами от $\approx N/4$ до $\approx N/8$ операция PUSHDOWN
 обрабатывает деревья высоты 2

Для узлов с номерами от $\approx N/8$ до $\approx N/16$ операция PUSHDOWN
 обрабатывает деревья высоты 3

Для узлов с номерами от $\approx N/16$ до $\approx N/32$ операция PUSHDOWN
 обрабатывает деревья высоты 4 и т.д.

Трудоемкость создания бинарного сбалансированного частично упорядоченного

дерева:

$$\sim \frac{N}{4} + \frac{2N}{8} + \frac{3N}{16} + \frac{4N}{32} + \dots < \frac{N}{4} \sum_{k=0}^{\infty} \frac{k+1}{2^k} = \frac{N}{4} \frac{1}{(1-1/2)^2} = N \quad (13)$$

и имеет линейный порядок сложности

Выполнение пирамидальной сортировки начинается с проецирования бинарного
 сбалансированного дерева в массив и его частичного упорядочивания. Затем при
 помощи операции DELETMIN из корня бинарного сбалансированного частично
 упорядоченного дерева поочередно извлекаются максимальное значение и в
 обратном порядке заносятся в выходной массив (т.е начиная с его конца) Таким
 образом трудоемкость пирамидальной сортировки оценивается величиной

$$\sim N + N \log_2 N \sim N \log_2 N \quad (14)$$

Данный алгоритм не требует дополнительного выделения оперативной памяти, поскольку входной массив, в который проецируется бинарное сбалансированное частично упорядоченное дерево, и выходной массив могут совпадать.

Однако непосредственное распараллеливание данного варианта алгоритма пирамидальной сортировки невозможно.

Внутренняя сортировка слиянием

Сортируемый массив $A = \{a_0, a_1, a_2, \dots, a_{N-1}\}$ (15)

На первой итерации метода упорядочиваем пары значений:

$(a_0, a_1), (a_2, a_3), (a_4, a_5), (a_6, a_7), \dots \sim N$ действий

На второй итерации используется алгоритм сортировки слиянием предварительно упорядоченных пар с предыдущей итерации, и упорядочиваются четверки :

$(a_0, a_1, a_2, a_3), (a_4, a_5, a_6, a_7), (a_8, a_9, a_{10}, a_{11}), \dots$, и набор $(a_{4\lfloor N/4 \rfloor - 1}, \dots, a_{N-1}) \sim N$ действий.

На третьей итерации используется алгоритм сортировки слиянием предварительно упорядоченных четверок с предшествующей итерации, и упорядочиваются восьмерки:

$(a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7, a_8), (a_9, a_{10}, a_{11}, a_{12}, a_{13}, a_{14}, a_{15}), \dots$, и набор $(a_{(8\lfloor \frac{N}{8} \rfloor - 1)}, \dots, a_{(N-1)}) \sim N$ действий.

И так далее. На последней итерации метода упорядочиваем весь массив алгоритмом сортировки слиянием предварительно упорядоченных половин массивов с предпоследней итерации, для чего требуется $\sim N$ действий.

Так как число итераций, очевидно, имеет порядок $\sim \log_2(N)$, то общая трудоемкость алгоритма асимптотически оценивается величиной

$$N \log_2 N$$

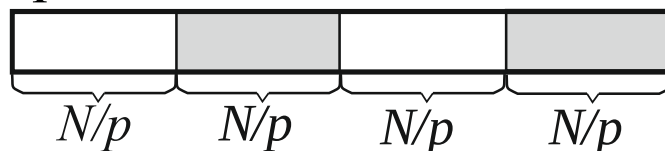
Упорядочивание различных пар, четверок, восьмерок и т.д. на первых итерациях алгоритма может быть выполнено независимо, и первые итерации метода сортировки слиянием могут быть очень хорошо распараллелены.

Однако распараллеливание завершающих итераций метода сортировки слиянием становится проблематичным.

При $p \in \mathbb{N}$ число используемых процессов характерное время оценивается как

$$\underbrace{N + \frac{N}{2} + \frac{N}{4} + \dots}_{\log_2 p} + \frac{1}{p} N (\log_2 N - \log_2 p) = 2N + \frac{N}{p} \log_2 \frac{N}{p} \quad (16)$$

Параллельный алгоритм сортировки массива A достаточно большой длины $N \gg 1$ $p \in \mathbb{N}$ — число процессоров, разделяем массив A на блоки длины $\approx N / p$.



и параллельно выполняем их сортировку последовательным быстрым алгоритмом пирамидальной сортировки или последовательным быстрым алгоритмом внутренней сортировки слиянием, что требует времени: $\sim \frac{N}{p} \log_2 \frac{N}{p}$ (17)

Далее, блочным алгоритмом четно-нечетной перестановкой с использованием лишь половины, т.е. $p / 2$, процессоров, сортируем массива A из предварительно упорядоченных блоков, что потребует времени $\sim 2N$ (оценка завышена).

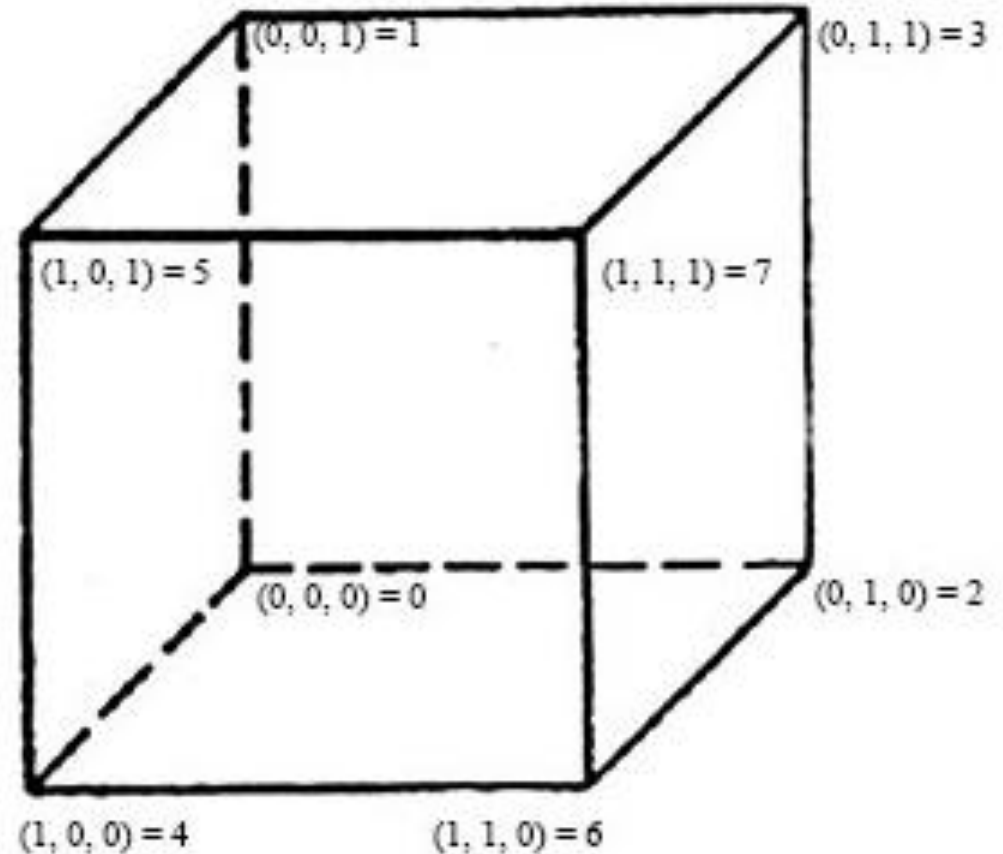
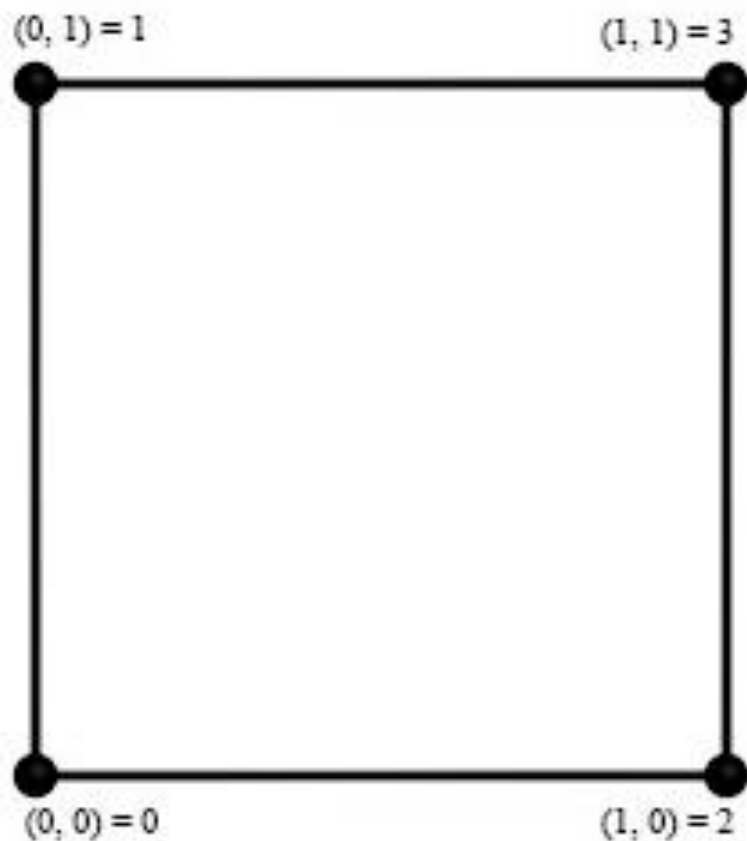
Общие затраты времени: $\sim 2N + \frac{N}{p} \log_2 \frac{N}{p}$ (18)

Ускорение: $S_p(N) = \frac{N \log_2 N}{2N + \frac{N}{p} \log_2 \frac{N}{p}} = \frac{\log_2 N}{\left(2 - \frac{1}{p} \log_2 p\right) + \frac{1}{p} \log_2 N} \xrightarrow{N \rightarrow \infty} p$ (19)

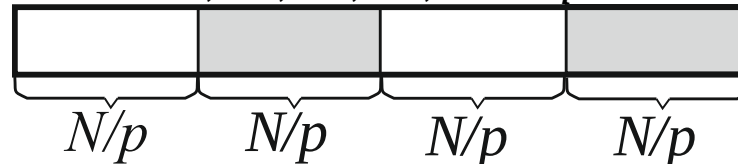
Ускорение $\approx p / 2$, равное примерно половине предельного, достигается при $N \approx 2^{2p}$ (при $p = 4$, $N \approx 236$; при $p = 6$, $N \approx 400$ при $p = 8$, $N \approx 6500$; при $p = 12$, $N \approx 4\,000\,000$ при $p = 16$, $N \approx 4\,000\,000\,000$), однако эти оценки излишне пессимистичны.

Параллельный вариант быстрой гиперсортровки Хоара.

Пусть число процессов $p = 2^m$. Каждое число j в диапазоне $0 \leq j \leq 2^m - 1$ может быть представлено в виде m -мерном битового вектора в двоичном представлении, а в m -мерном пространстве точки, соответствующие данным векторам, располагается в вершинах некоторого гиперкуба. Заметим, что любой номер с первой координатой 0 в гиперкубе строго меньше, чем номер с первой координатой 1 в гиперкубе.



На первой фазе алгоритма, аналогично предведущему, разбиваем исходный массив A длины N на несколько блоков $A_0, A_1, A_2, A_3, \dots, A_p$ примерно равной длины $\approx N / p$.



Параллельно при помощи $p = 2^m$ процессоров выполняем их сортировку последовательным алгоритмом пирамидальной сортировки или последовательным алгоритмом внутренней сортировки слиянием. Время: $\sim \frac{N}{p} \log_2 \frac{N}{p}$ (20)

На второй фазе алгоритма используются $p / 2 \approx 2^{m-1}$ процессоров.

По номеру j блока A_j сопоставляем блоку некоторую вершину гиперкуба. Блоки могут изменять длину и расположение в оперативной памяти, однако суммарная длина всех блоков $\approx N$ и порядок их расположения остаются неизменными.

Выбираем ведущее значение. Образует $p / 2 = 2^{m-1}$ пар из блоков, для которых в гиперкубе отличаются лишь значения первой координаты. На $p / 2 = 2^{m-1}$ процессорах выполняем сортировку слиянием упорядоченных массивов из составляющих пары блоков, и полученные в результате слияния массивы снова разделяем на 2 блока так, чтобы в блоки со значением 0 первой координаты в гиперкубе попали все значения, меньшие ведущего, а в блоки со значением 1 первой координаты в гиперкубе попали все остальные значения («сравнить и разделить»).

Исходный гиперкуб размерности m оказывается разделен на 2 гиперкуба размерности $m-1$, к которым рекурсивно применяется процедура гиперсортировки. После $m-1$ итераций размерность гиперкубов сократится до 1, что соответствует одной паре блоков.

Заметим, что после деления некоторого гиперкуба на 2 гиперкуба меньшей размерности суммарная длина блоков в каждом из гиперкубов меньшей размерности остается неизменной.

В наилучшем случае, когда после сортировки и слияния блоков данные снова разделяются на примерно равные блоки, собственно гиперсортировка будет

выполняться за время
$$\frac{m \cdot 2N}{p} = \frac{2N \log_2 p}{p} \quad (21)$$

Общее время выполнения параллельной версии алгоритма

$$\approx \frac{N}{p} \log_2 \frac{N}{p} + \frac{2N \log_2 p}{p} \approx \frac{N}{p} (\log_2 p + \log_2 N) \quad (3.44)$$

Ускорение
$$S_p(N) = \frac{N \log_2 N}{\frac{N}{p} (\log_2 p + \log_2 N)} = p \frac{\log_2 N}{\log_2 p + \log_2 N} \xrightarrow{N \rightarrow \infty} p \quad (22)$$

Ускорение $\frac{3}{4}$ от предельного достигается при $N \approx p^3$ (при $p = 32$, $N \approx 32000$), то есть достаточно быстро. Однако эта оценка достаточно оптимистична.

В неблагоприятном случае, когда на 1-й итерации размер половины блоков сократится до одного элемента, а размер другой половины блоков увеличится примерно в 2 раза, затем размер 1/4 от общего числа блоков снова сокращается до одного элемента, а размер другой 1/4 от общего числа блоков снова возрастет примерно в 2 раза, и т.д., собственно гиперсортировка выполняется за время

$$\frac{N}{p} + \frac{2N}{p} + \dots + N = N + \frac{1}{2}N + \frac{1}{4}N + \dots + 2^{-m}N < 2N$$

РЕМ. STL для C++ реализует т.н. интроспективную сортировку.

Сначала к массиву рекурсивно начинает применяться сортировка Хоара.

После первого сбоя сортировки Хоара некоторого фрагмента, вызванного неправильным выбором элемента-разделителя, для дальнейшей его сортировки применяется пирамидальная сортировка.

Т.к. данный алгоритм характеризуется рекурсивным параллелизмом (рекурсивный параллелизм приводит к обходу ветвей дерева, и это может быть выполнено независимо, т.е. параллельно), он хорошо распараллеливается.

Параллельные реализации в MS Concurrency Runtime.

Время исполнения базового алгоритма

$$\sim 2N + \frac{N}{p} \log_2 \frac{N}{p}$$

Время выполнения алгоритма с резервированием дополнительной памяти

$$\sim \frac{N}{p} \log_2 \frac{N}{p}$$

Поиск k первых наибольших значений из N при $1 \ll k \ll N$

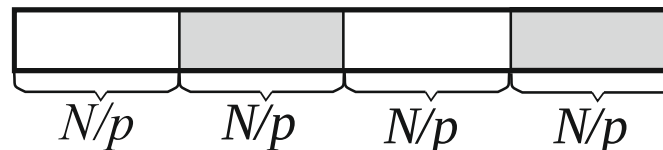
(например: top 1000 из 100000000) в последовательном варианте на основе первых k этапов обычной пузырьковой сортировки дает оценку сложности $\sim kN$

В последовательном варианте: из сортируемого массива длины N создается сбалансированное бинарное частично упорядоченное дерево. Трудоемкость $\sim N$.

Затем из созданного дерева последовательно извлекаются с удалением k первых наибольших значений. Трудоемкость $\sim k \log_2 N$.

В последовательном варианте, трудоемкость данного алгоритма $\sim N + k \log_2 N$.

В параллельном варианте, массив разбивается на $p \in \mathbb{N}$ (p – число процессов) частей длины $\approx N / p$.



Для каждой части массива параллельно создается сбалансированное бинарное частично упорядоченное дерево. Затраты времени $\sim N / p$.

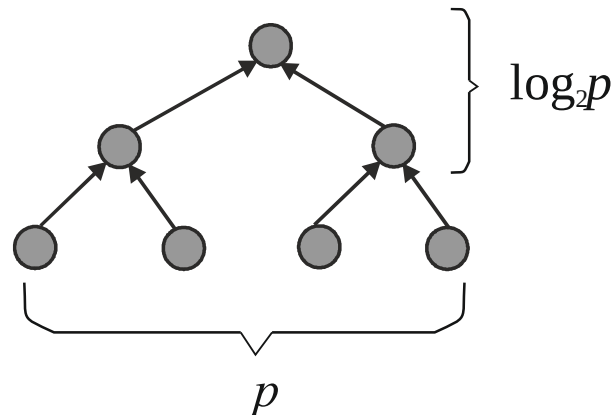
Параллельно из созданных деревьев поочередно извлекаются с удалением по k наибольших значений с затратами времени $\sim k \log_2 \frac{N}{p}$.

Далее из p предварительно упорядоченных наборов по k извлеченных значений выбирается k максимальных значений.

Например, слиянием первых двух упорядоченных по убыванию наборов длины k находится упорядоченный по убыванию набор длины k ; его слиянием с третьим упорядоченным по убыванию набором длины k снова находится упорядоченный по убыванию набор длины k и т. д.

Последовательное выполнение операций слияния p наборов приводит к затратам времени $\sim pk$.

Распараллеливание на основе аналога каскадной схемы приводит к оценке $\sim k \log_2 p$



Т. е., время выполнения параллельной версии

$$T_p(N) \sim \frac{N}{p} + k \log_2 \frac{N}{p} + k \log_2 p = \frac{N}{p} + k \log_2 N \quad (23)$$

Соответственно, ускорение вычислительного процесса:

$$S_p(N) = \frac{T_1(N)}{T_p(N)} = \frac{N + k \log_2 N}{N / p + k \log_2 N} \xrightarrow{N \rightarrow \infty} p \quad (24)$$